

Pairing-free Blind Signatures in the ROM

Towards secure schemes from the DL assumption

Michael Klooß

KIT

Russell W. F. Lai

Aalto University

Michael Reichle

ETH Zurich



ETH zürich

Prelims and Motivation

Recap

Digital Signatures

- Overview

Recap

Digital Signatures

- Overview
 - sign m with secret key → 
 - verify  for m with verification key

Recap

Digital Signatures

- Overview
 - sign m with secret key → 
 - verify  for m with verification key
 - *integrity*: message is unaltered

Recap

Digital Signatures

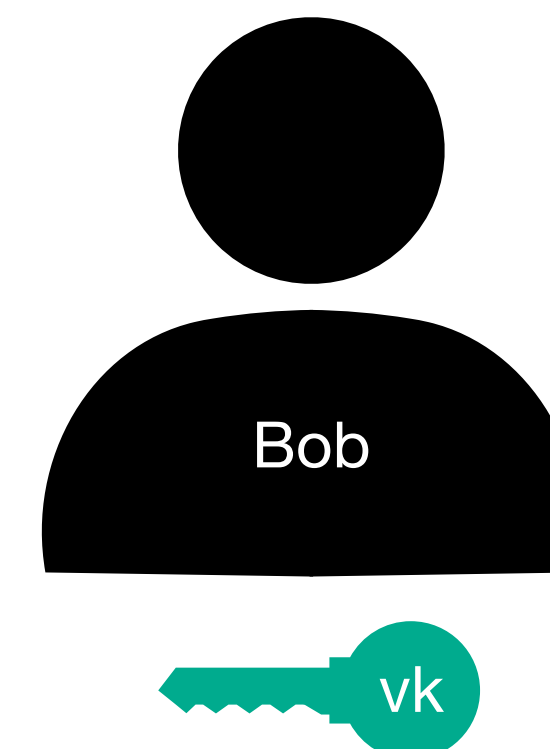
- Overview

- sign m with secret key → 
- verify  for m with verification key
 - *integrity*: message is unaltered

- **Example:** Schnorr signatures

Running Example

Electronic Cash [Cha82]

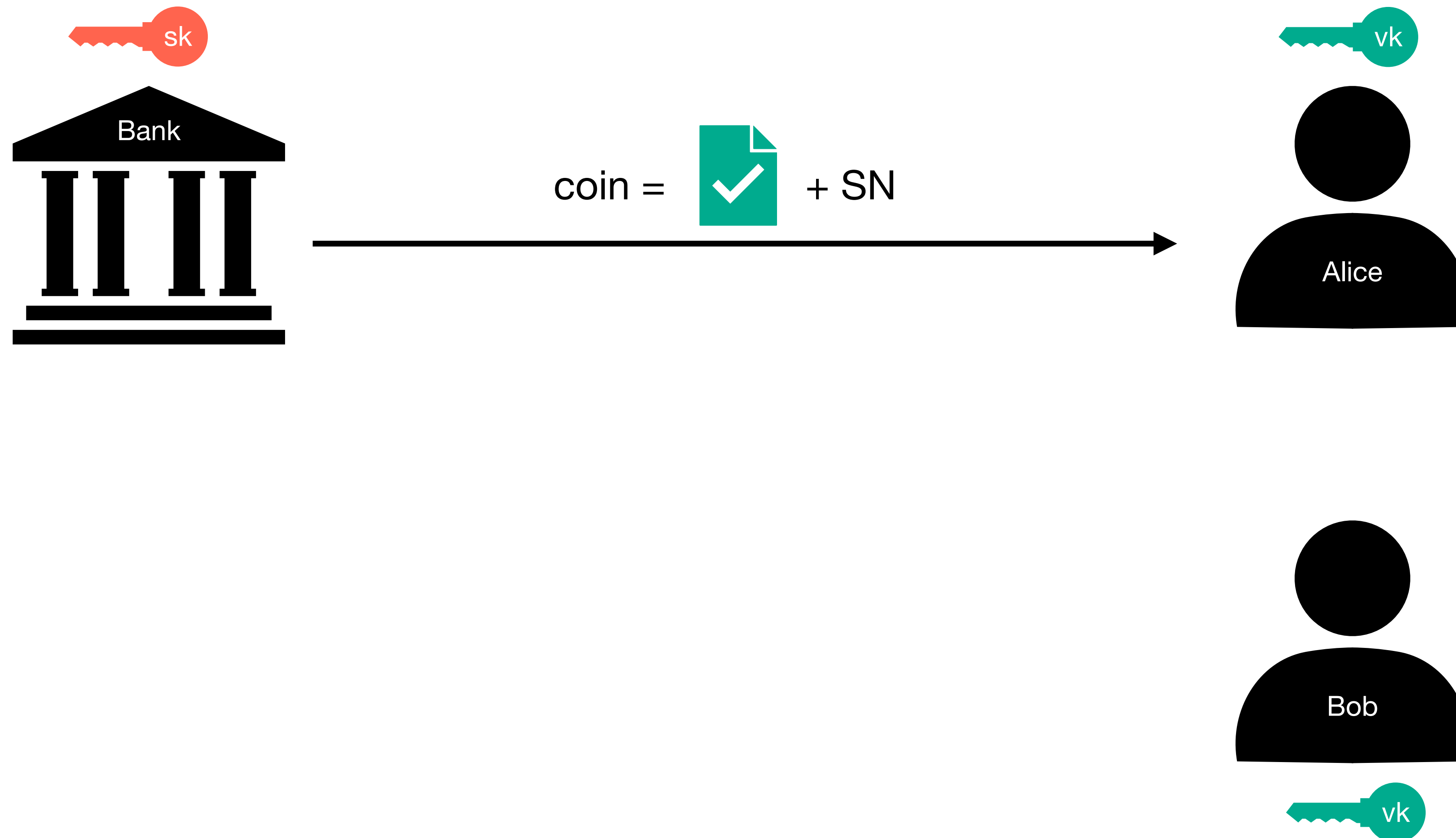


Running Example

Electronic Cash [Cha82]



- SN is a *random* serial number chosen by Alice

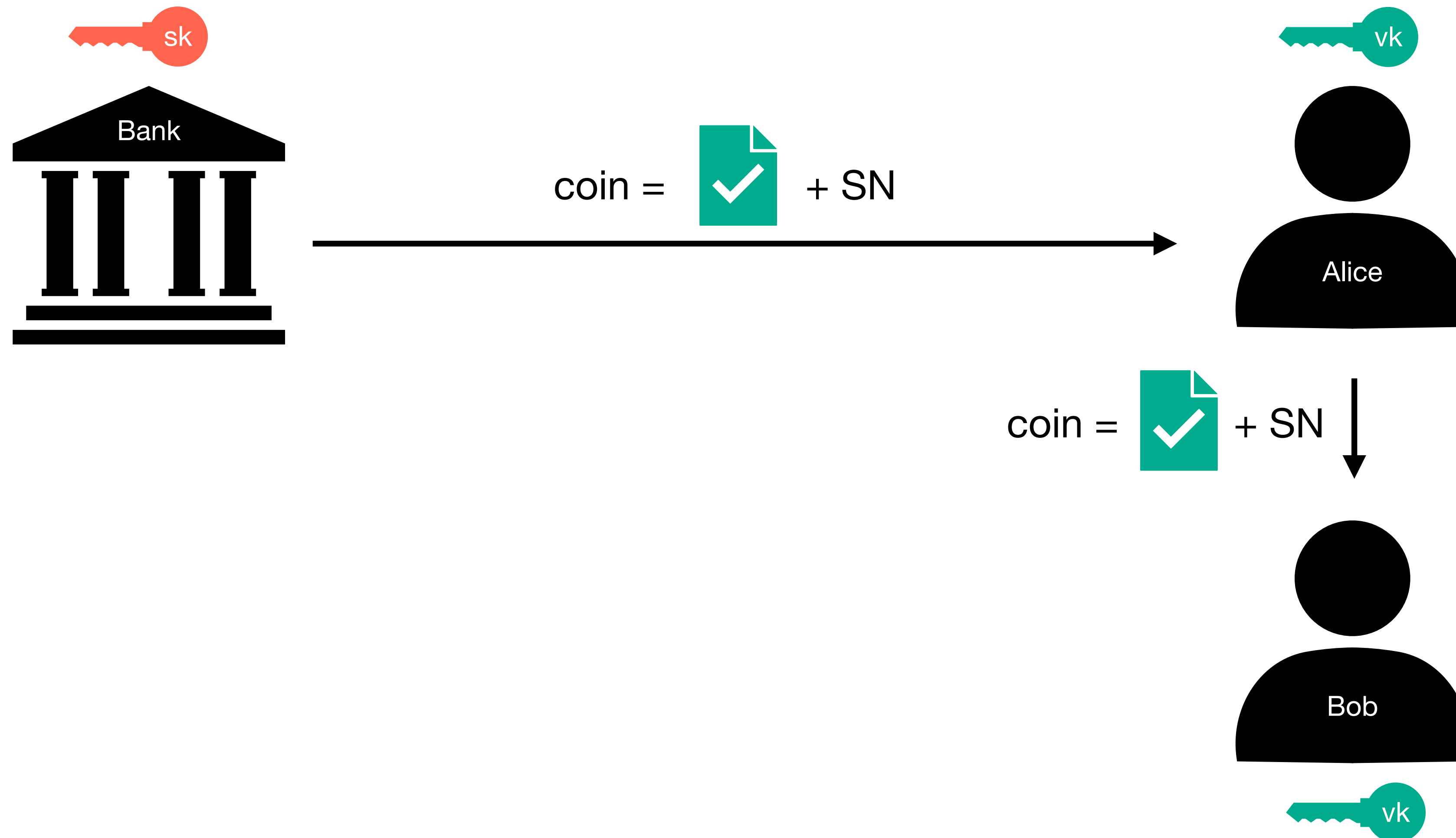


Running Example

Electronic Cash [Cha82]



- SN is a *random* serial number chosen by Alice

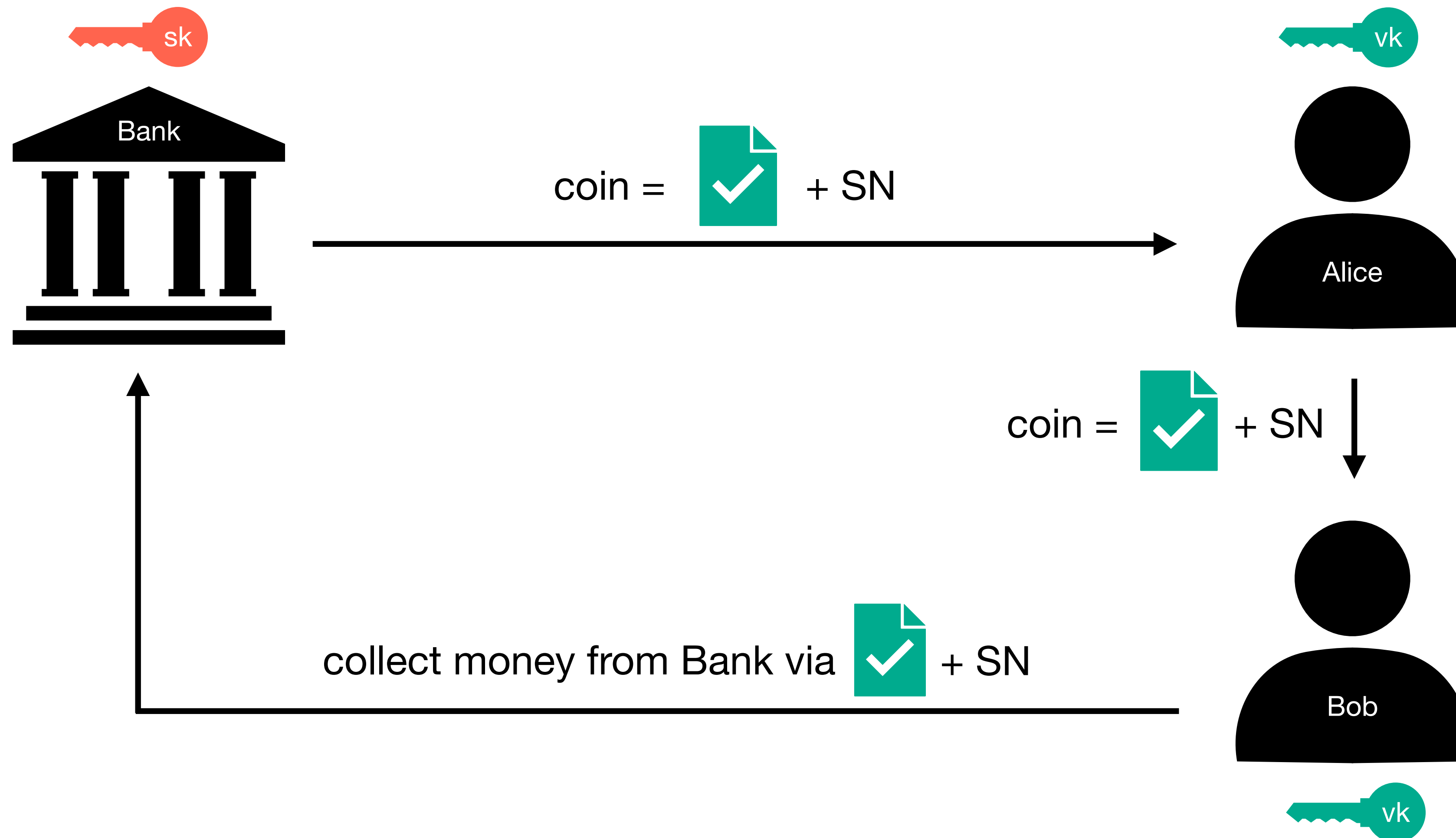


Running Example

Electronic Cash [Cha82]



- SN is a *random* serial number chosen by Alice
- The bank keeps track of used serial numbers

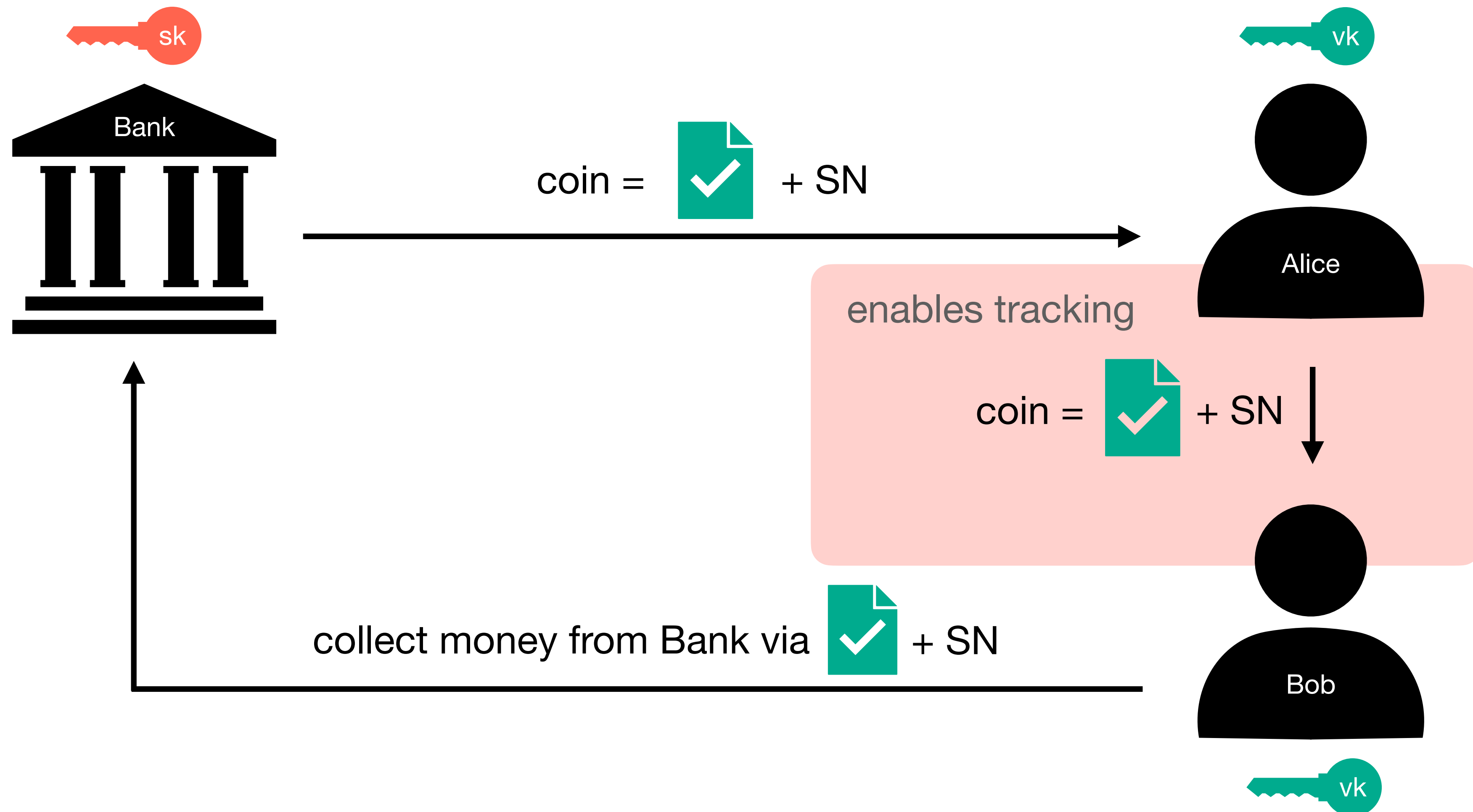


Running Example

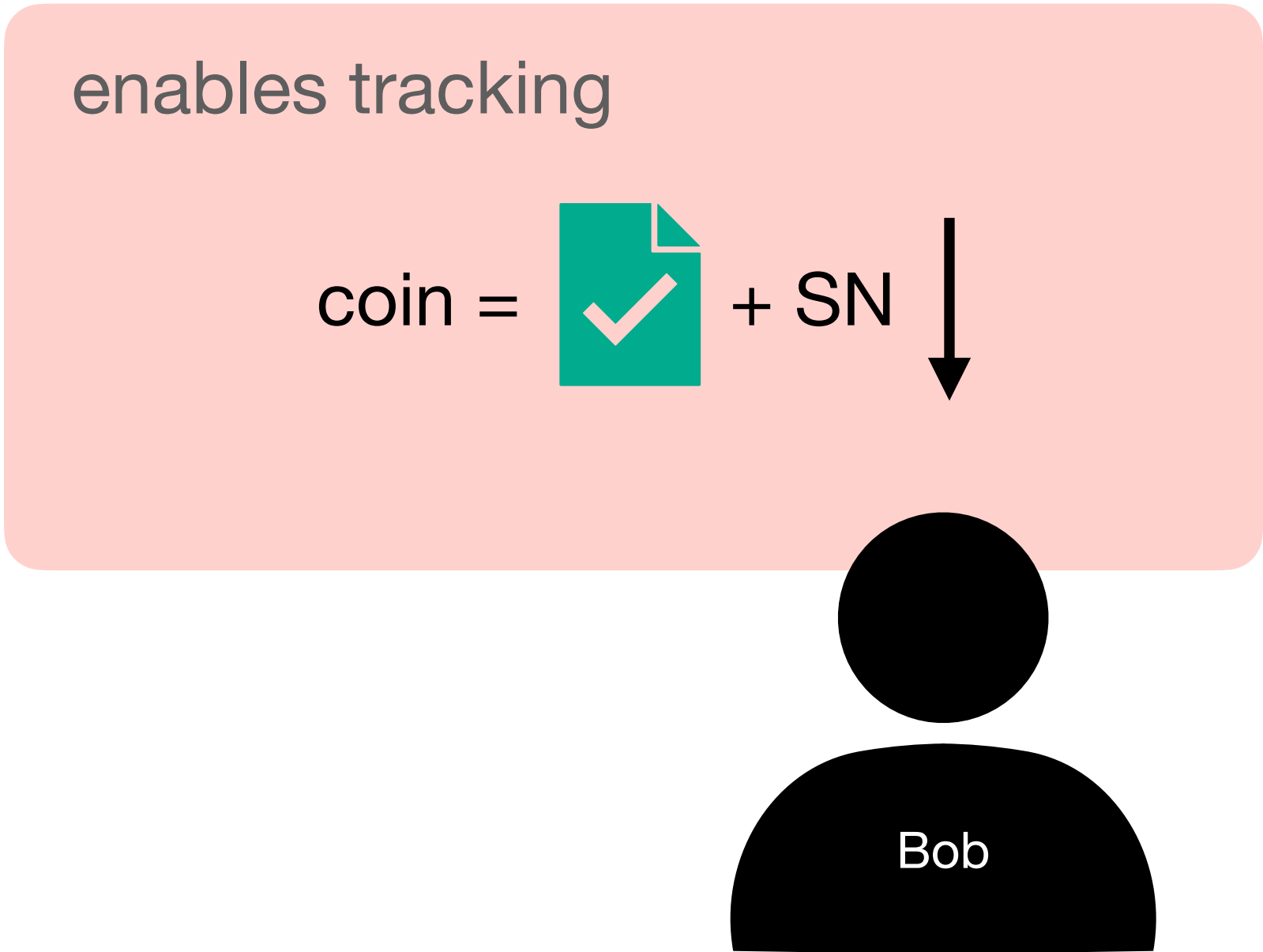
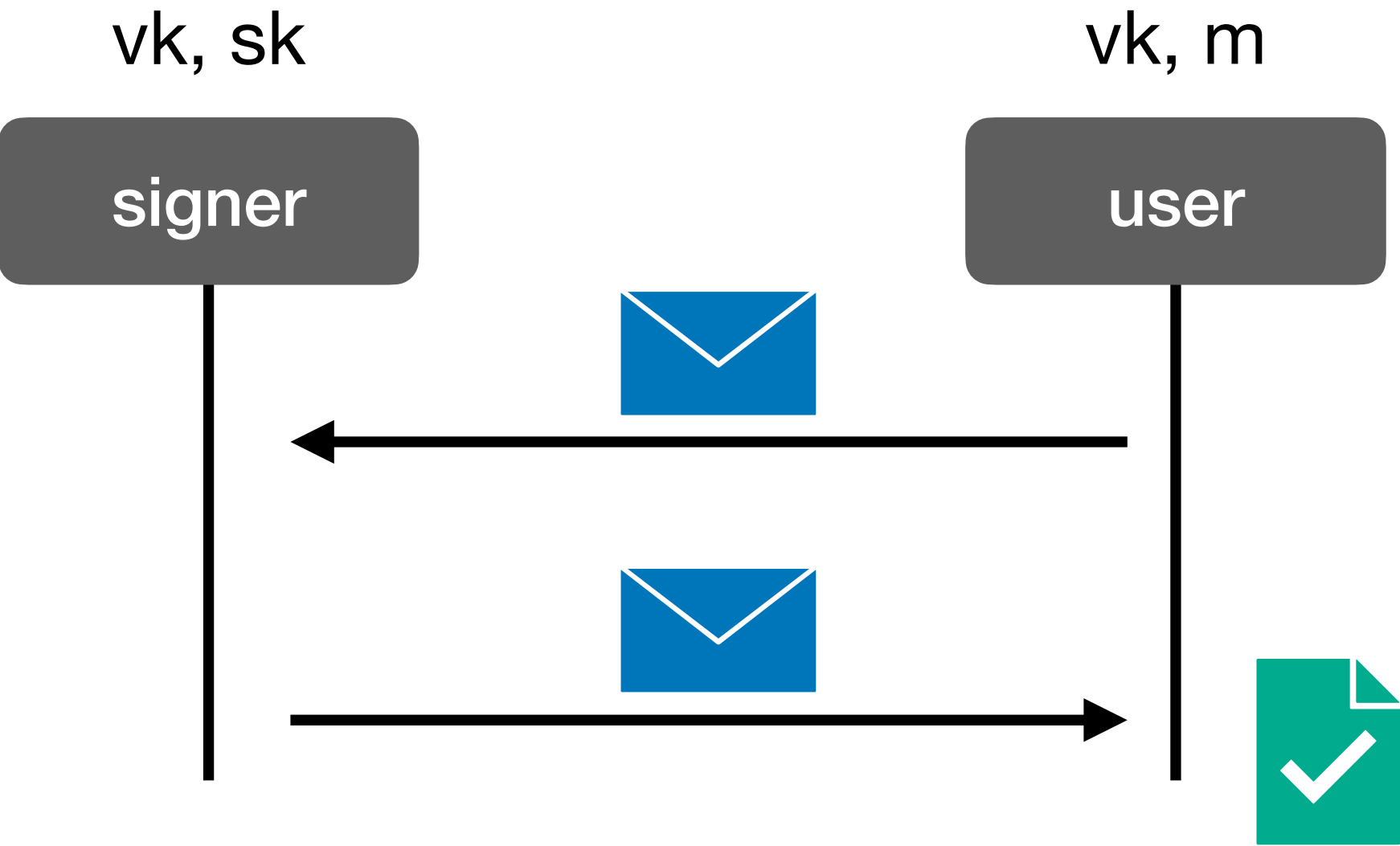
Electronic Cash [Cha82]



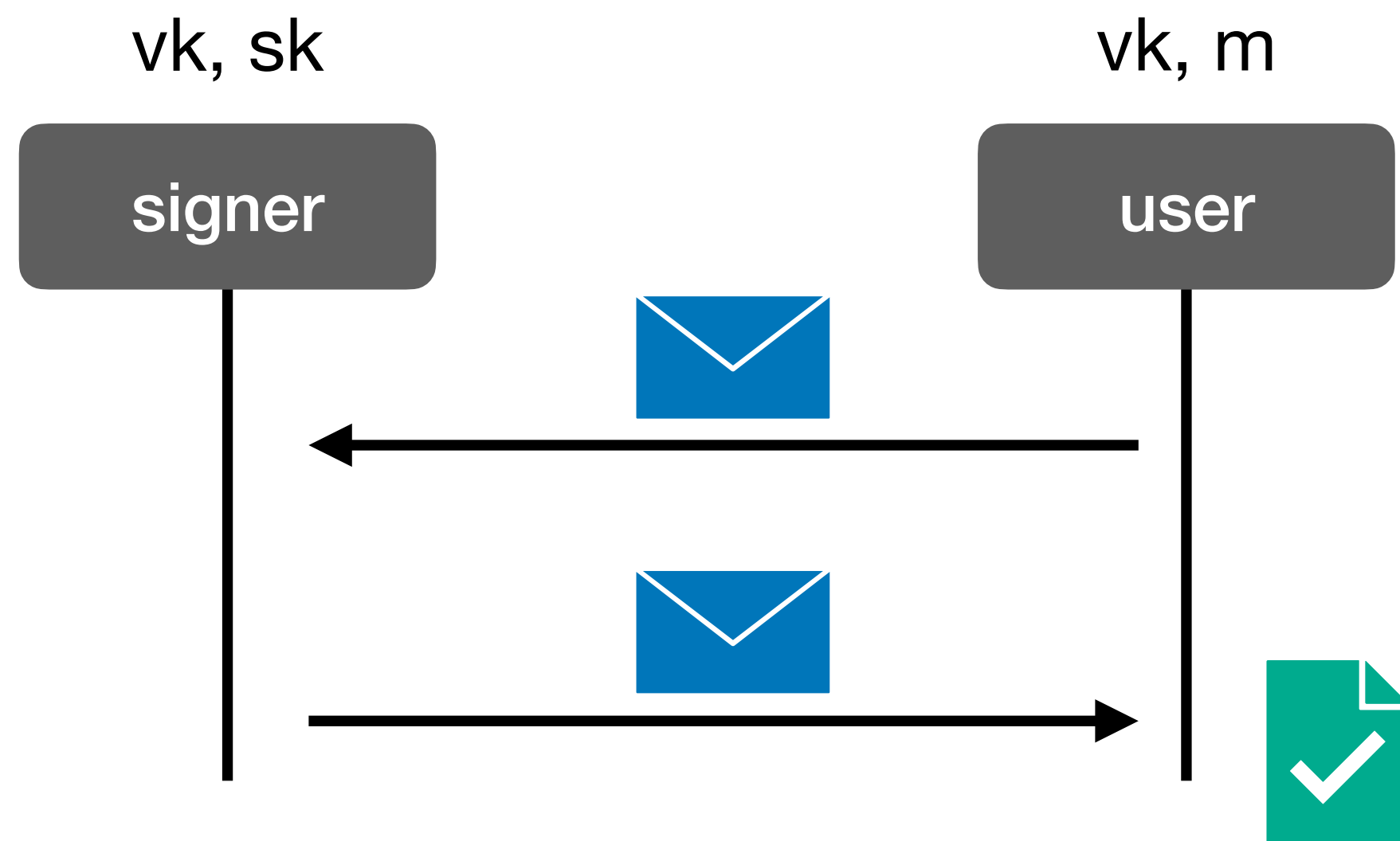
- SN is a *random* serial number chosen by Alice
- The bank keeps track of used serial numbers



Blind Signatures

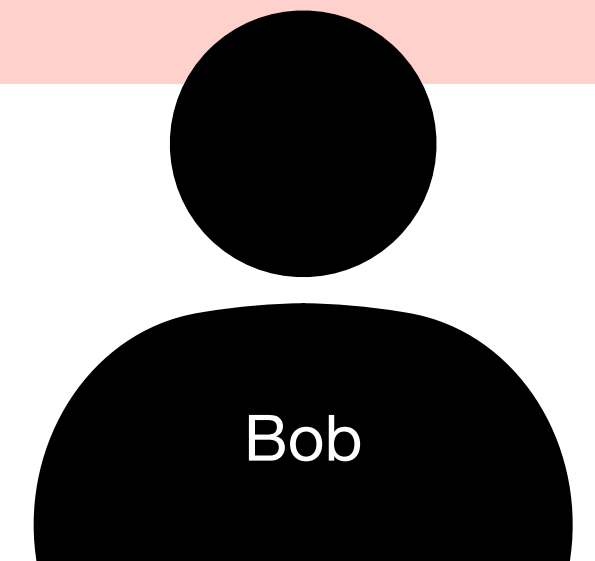


Blind Signatures



enables tracking

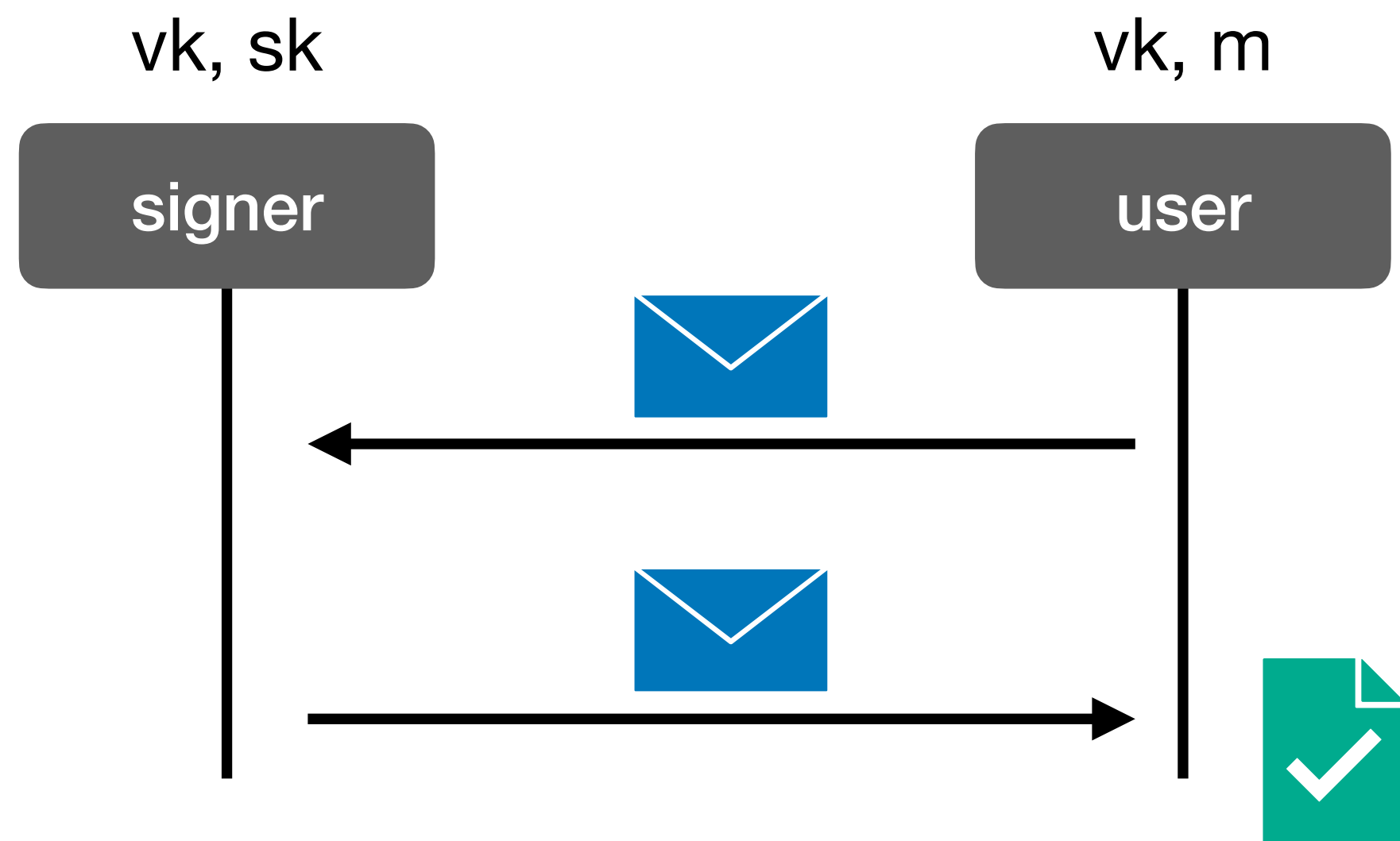
$$\text{coin} = \text{✓} + \text{SN} \downarrow$$



Correctness:

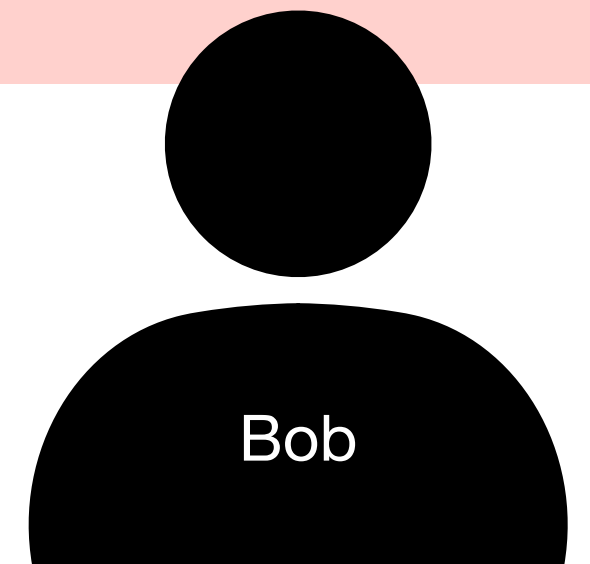
- Honest signatures verify

Blind Signatures



enables tracking

$$\text{coin} = \text{✓} + \text{SN} \downarrow$$



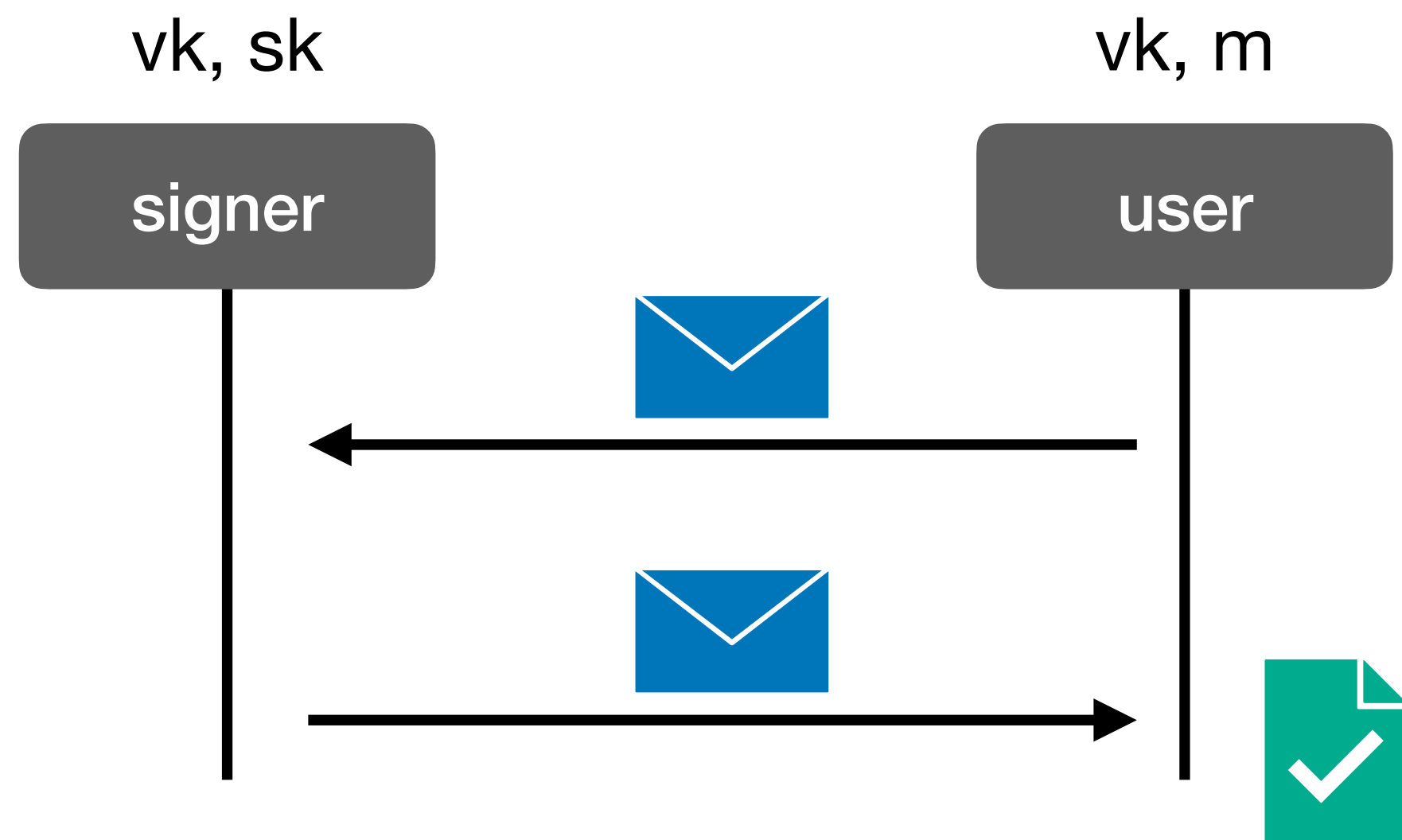
Correctness:

- Honest signatures verify

Blindness:

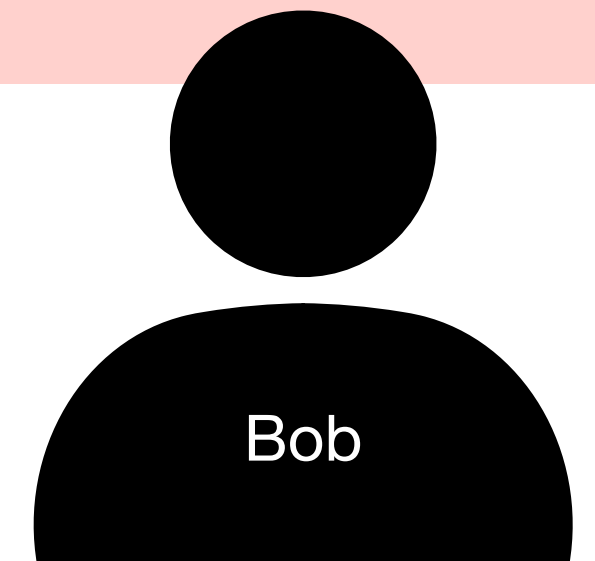
- cannot link ✓ and m to the signing session

Blind Signatures



enables tracking

$$\text{coin} = \text{✓} + \text{SN} \downarrow$$



Correctness:

- Honest signatures verify

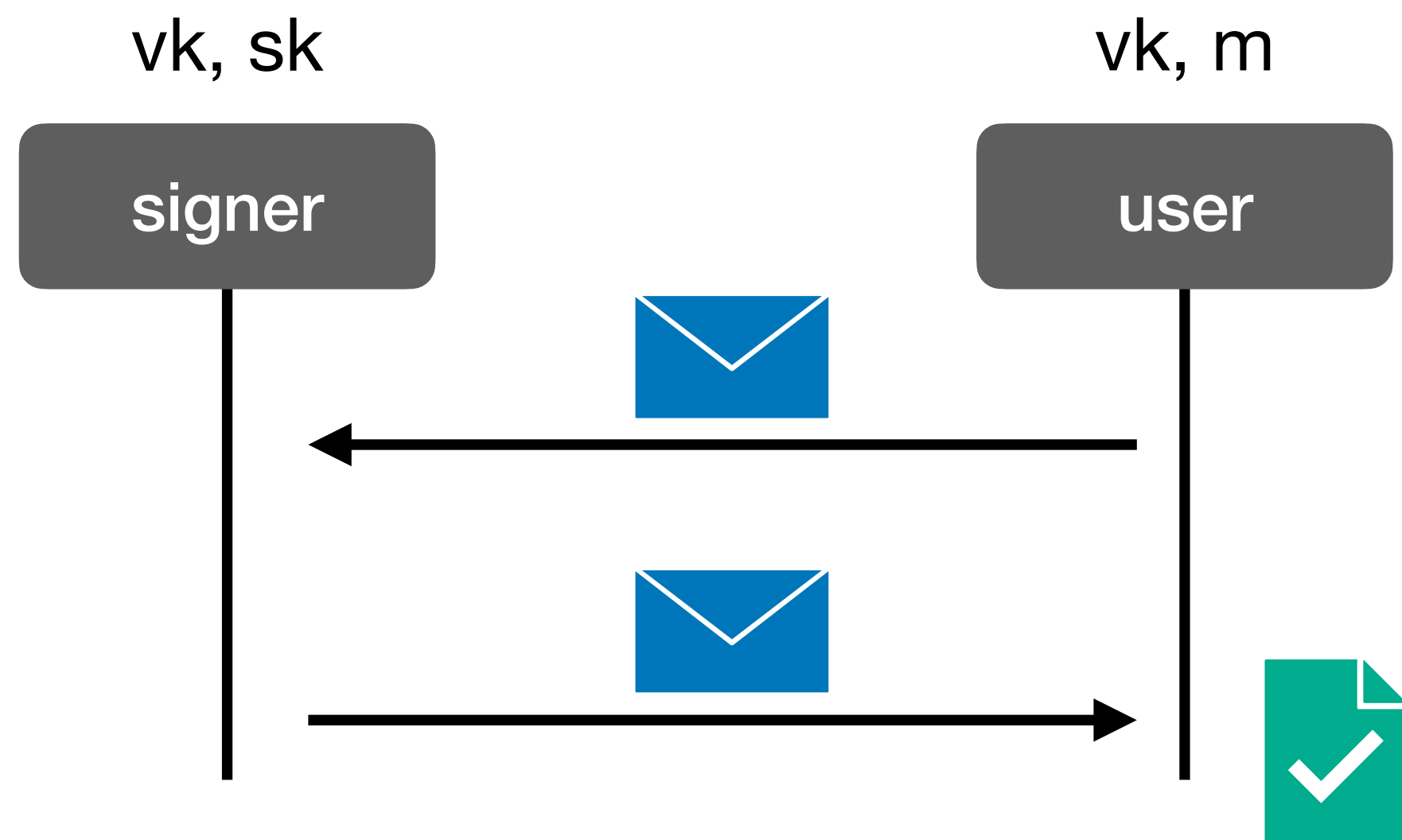
Blindness:

- cannot link  and m to the signing session

Unforgeability:

- must interact with signer to obtain  on m

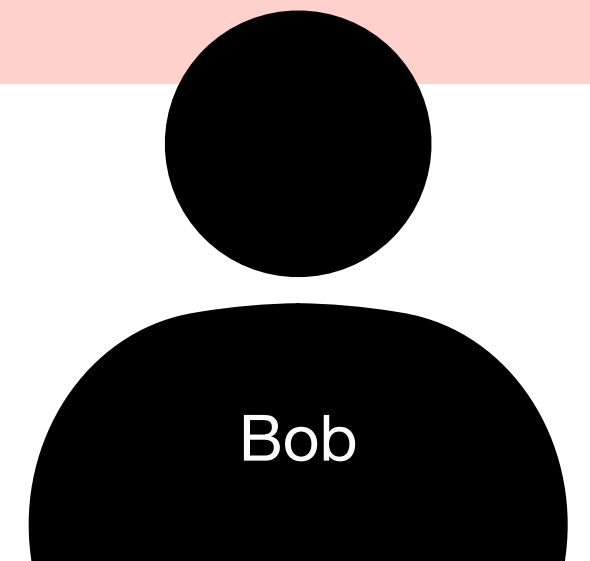
Blind Signatures



- electronic cash [SNB]
- privacy pass [CloudFlare]
- VPNs [Google]

enables tracking

$$\text{coin} = \text{✓} + \text{SN} \downarrow$$



Correctness:

- Honest signatures verify

Blindness:

- cannot link ✓ and m to the signing session

Unforgeability:

- must interact with signer to obtain ✓ on m

Recap

Σ -protocols

relation $(X, W) \in R$

witness W

prover

statement X

verifier

Recap

Σ -protocols

relation $(\mathbb{X}, \mathbb{W}) \in R$

witness $\mathbb{W}$

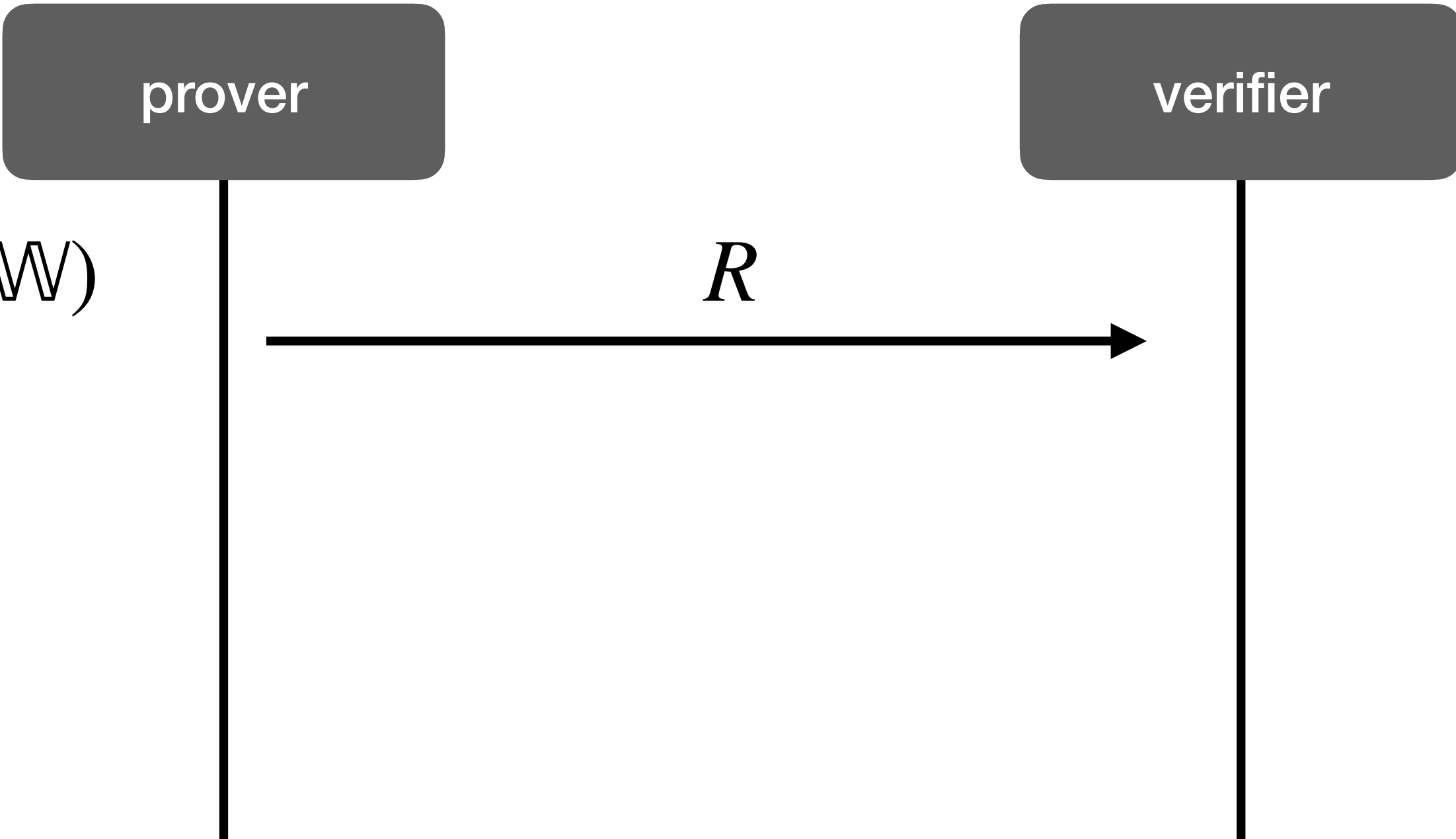
statement $\mathbb{X}$

prover

verifier

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R



```
graph LR; P[prover] --- V[verifier]; P -- R --> V;
```

Recap

Σ -protocols

relation $(\mathbb{X}, \mathbb{W}) \in R$

witness $\mathbb{W}$

statement $\mathbb{X}$

prover

verifier

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

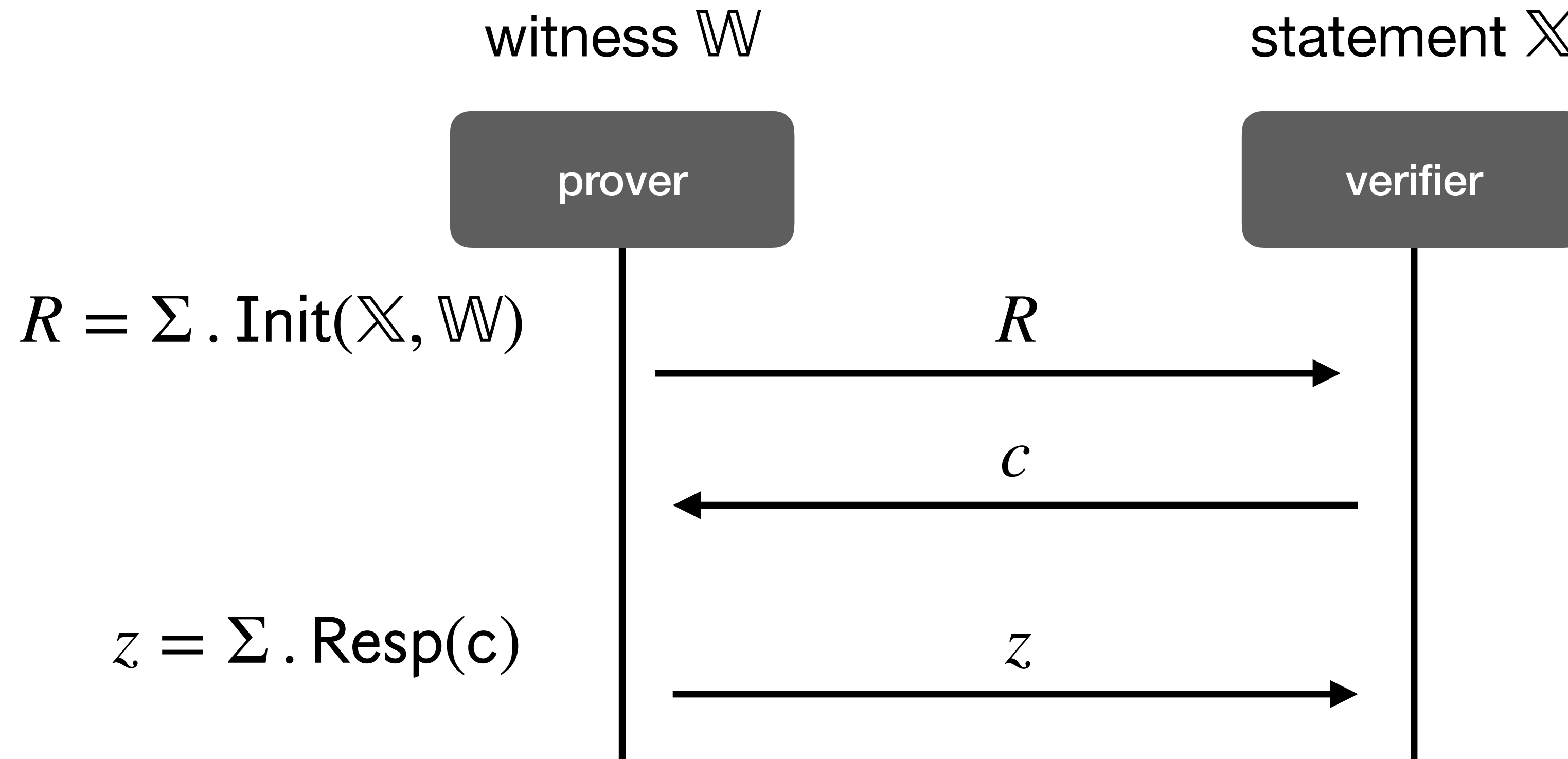
R

c

Recap

Σ -protocols

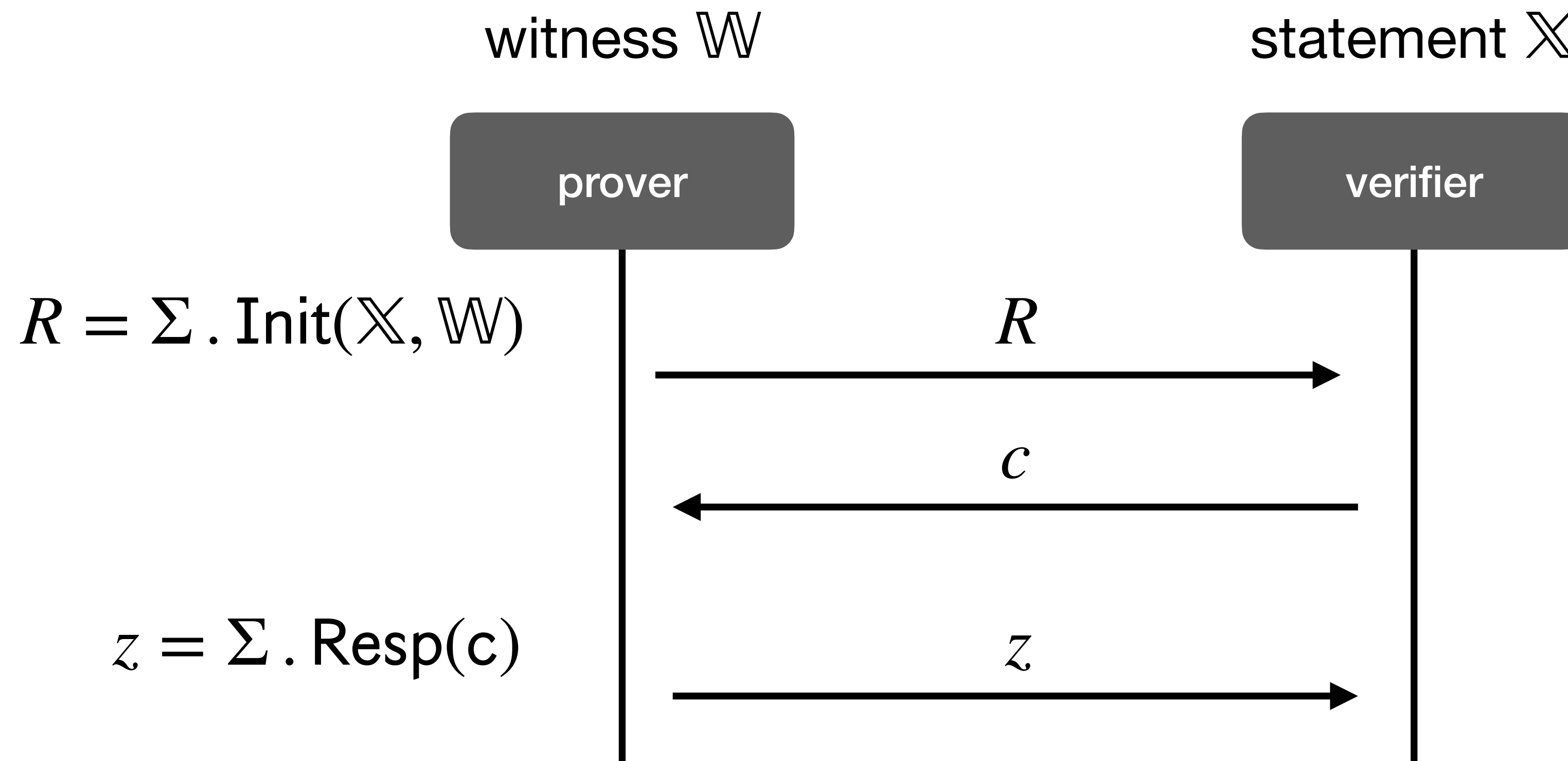
relation $(\mathbb{X}, \mathbb{W}) \in R$



Recap

Σ -protocols

relation $(\mathbb{X}, \mathbb{W}) \in R$

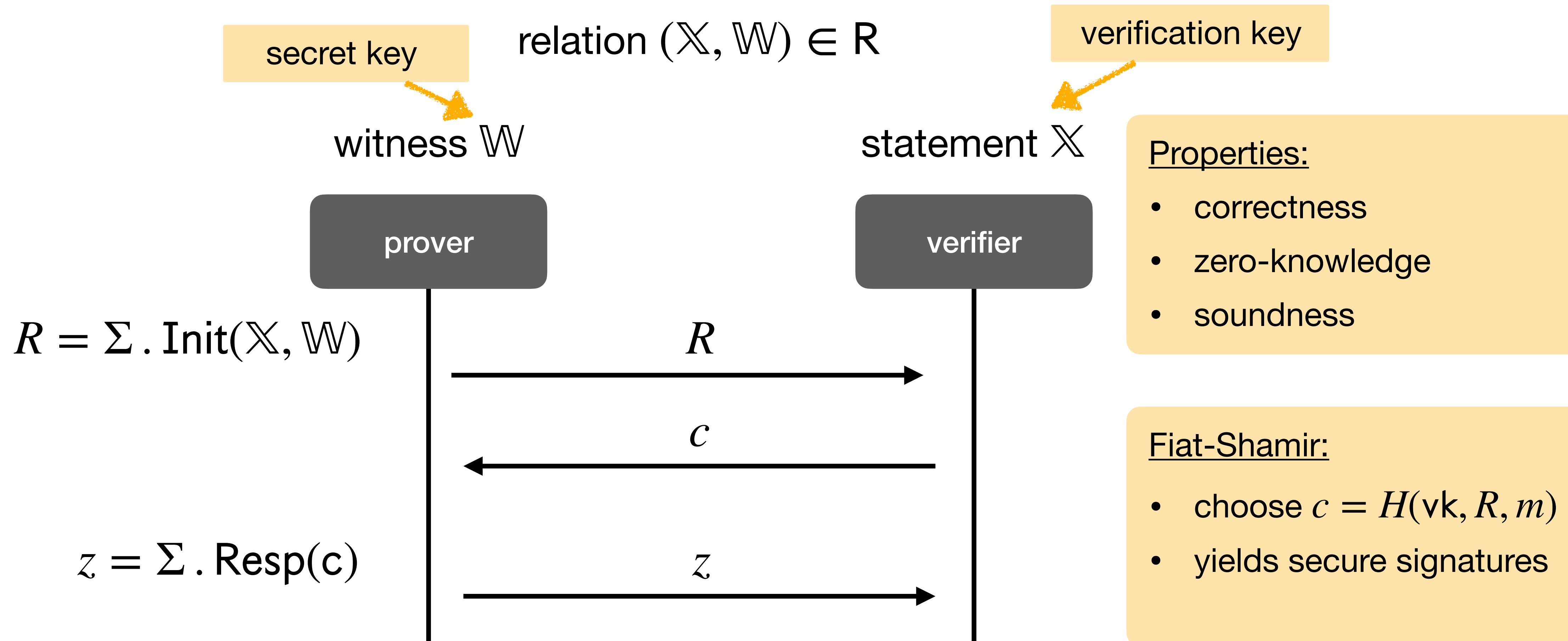


Properties:

- correctness
- zero-knowledge
- soundness

Recap

Σ -protocols



Recap

Σ -protocols

Schnorr: DL relation
 $\mathbb{X} = (G, X) \in \mathbb{G}^2$
 $\mathbb{W} = x \in \mathbb{Z}_p$ such that $xG = X$

secret key

relation $(\mathbb{X}, \mathbb{W}) \in R$

verification key

witness $\mathbb{W}$

statement $\mathbb{X}$

prover

verifier

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

c

$z = \Sigma . \text{Resp}(c)$

z

Properties:

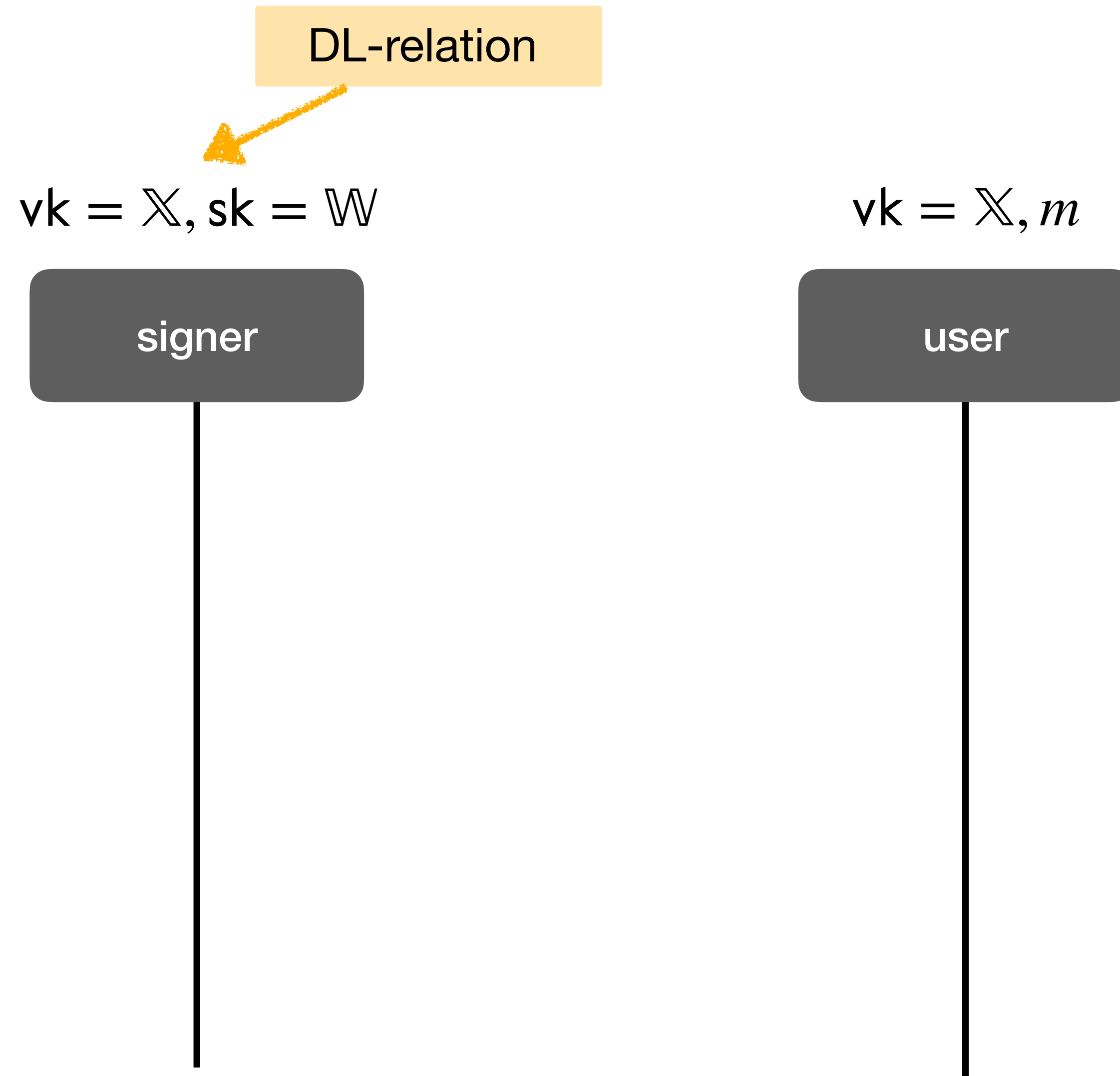
- correctness
- zero-knowledge
- soundness

Fiat-Shamir:

- choose $c = H(\text{vk}, R, m)$
- yields secure signatures

Insecure Example

Blind Schnorr



Insecure Example

Blind Schnorr

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

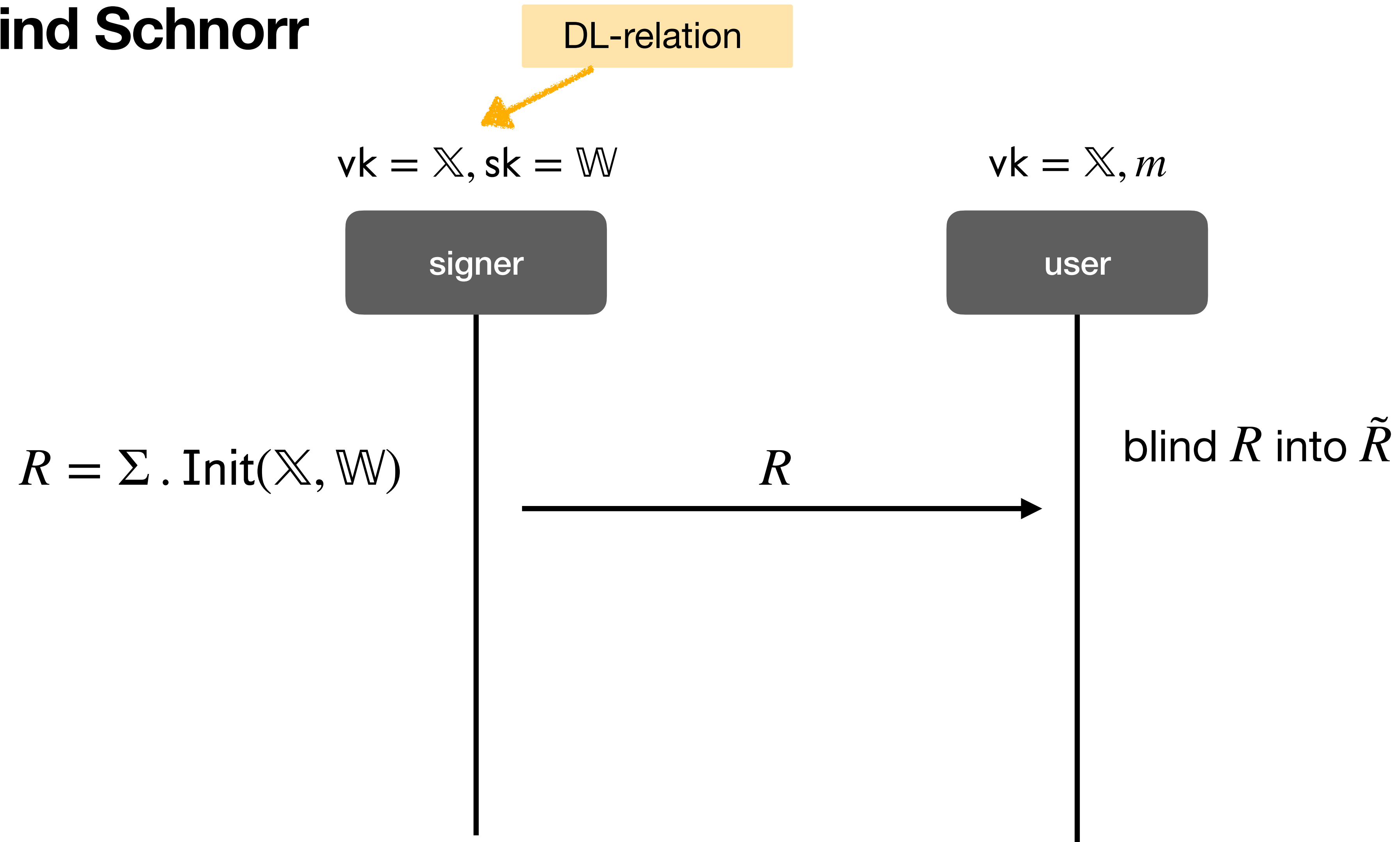
user

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

Insecure Example

Blind Schnorr



Insecure Example

Blind Schnorr

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

user

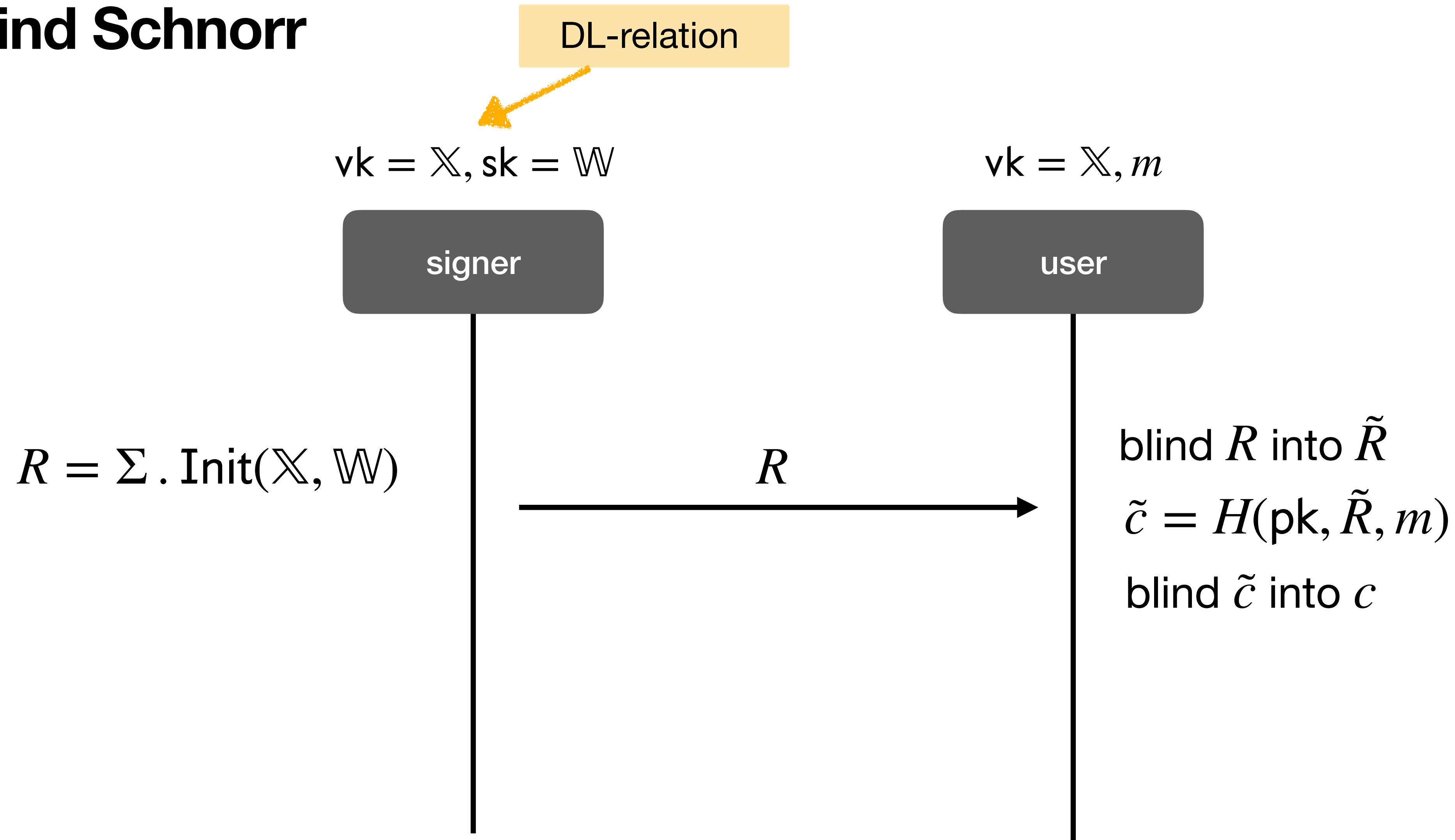
$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

blind R into $\tilde{R}$
 $\tilde{c} = H(pk, \tilde{R}, m)$

Insecure Example

Blind Schnorr



Insecure Example

Blind Schnorr

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

user

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

blind R into $\tilde{R}$
 $\tilde{c} = H(pk, \tilde{R}, m)$
blind $\tilde{c}$ into c

c

Insecure Example

Blind Schnorr

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

user

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

blind R into $\tilde{R}$
 $\tilde{c} = H(pk, \tilde{R}, m)$
blind $\tilde{c}$ into c

c

$z = \Sigma . \text{Resp}(c)$

z

Insecure Example

Blind Schnorr

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

user

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

blind R into $\tilde{R}$
 $\tilde{c} = H(pk, \tilde{R}, m)$

blind $\tilde{c}$ into c

c

blind z into $\tilde{z}$

$z = \Sigma . \text{Resp}(c)$

z

Insecure Example

Blind Schnorr

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

user

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

blind R into $\tilde{R}$
 $\tilde{c} = H(pk, \tilde{R}, m)$

blind $\tilde{c}$ into c

c

$z = \Sigma . \text{Resp}(c)$

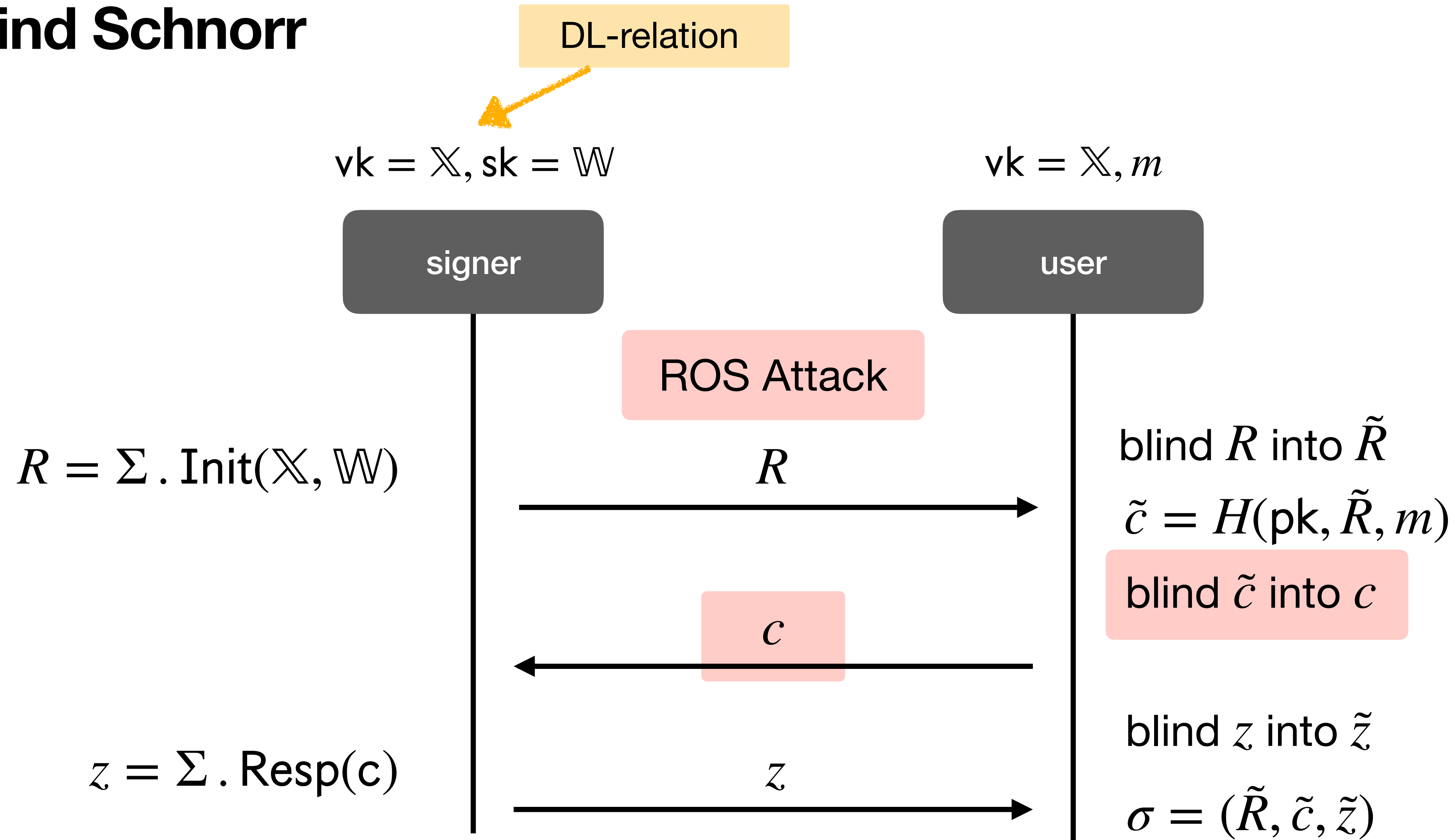
z

blind z into $\tilde{z}$

$\sigma = (\tilde{R}, \tilde{c}, \tilde{z})$

Insecure Example

Blind Schnorr



Insecure Example

Blind Schnorr



ignore blinding for the rest of the talk

DL-relation

$vk = \mathbb{X}, sk = \mathbb{W}$

signer

$vk = \mathbb{X}, m$

user

ROS Attack

$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$

R

blind R into $\tilde{R}$

$\tilde{c} = H(pk, \tilde{R}, m)$

blind $\tilde{c}$ into c

c

blind z into $\tilde{z}$

$\sigma = (\tilde{R}, \tilde{c}, \tilde{z})$

$z = \Sigma . \text{Resp}(c)$

z

Motivation



- ROM = Random Oracle Model
- Heuristic model for hash functions H

- **Digital Signatures:** Schnorr is secure under DL in the ROM

Motivation



- ROM = Random Oracle Model
- Heuristic model for hash functions H

- **Digital Signatures:** Schnorr is secure under DL in the ROM
- **Blind Signatures:** no known solution from DL in the ROM so far...

Motivation



- ROM = Random Oracle Model
- Heuristic model for hash functions H

- **Digital Signatures:** Schnorr is secure under DL in the ROM
- **Blind Signatures:** no known solution from DL in the ROM so far...
- **Known Tradeoffs:** how to build blind signatures the ROM

Motivation



- ROM = Random Oracle Model
- Heuristic model for hash functions H

- **Digital Signatures:** Schnorr is secure under DL in the ROM
- **Blind Signatures:** no known solution from DL in the ROM so far...
- **Known Tradeoffs:** how to build blind signatures the ROM
 - security under DDH

Motivation



- ROM = Random Oracle Model
- Heuristic model for hash functions H

- **Digital Signatures:** Schnorr is secure under DL in the ROM
- **Blind Signatures:** no known solution from DL in the ROM so far...
- **Known Tradeoffs:** how to build blind signatures the ROM
 - security under DDH
 - limited security under DL

Motivation



- ROM = Random Oracle Model
- Heuristic model for hash functions H

- **Digital Signatures:** Schnorr is secure under DL in the ROM
- **Blind Signatures:** no known solution from DL in the ROM so far...
- **Known Tradeoffs:** how to build blind signatures the ROM
 - security under DDH
 - limited security under DL
 - security under DL *and* AGM

Recap of Prior Work

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:
 - (i) can sign all messages $m \neq m^*$
 - (ii) signature on m is hard to compute

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:
 - (i) can sign all messages $m \neq m^*$
 - (ii) signature on m is hard to compute
 - $\mathbb{X}$ is compact and linear $\rightarrow$ efficient blind issuance

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:
 - (i) can sign all messages $m \neq m^*$
 - (ii) signature on m is hard to compute
 - $\mathbb{X}$ is compact and linear $\rightarrow$ efficient blind issuance
- **Statement $\mathbb{X}$:** inequality of encrypted messages

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:
 - (i) can sign all messages $m \neq m^*$
 - (ii) signature on m is hard to compute
 - $\mathbb{X}$ is compact and linear $\rightarrow$ efficient blind issuance
- **Statement $\mathbb{X}$:** inequality of encrypted messages

$C := C^* - \text{Enc}(\text{pk}, m; 0)$ does not encrypt 0

Recap of [KR25]

Blind Signatures from Puncturing

- **Idea:** craft tailored statement $\mathbb{X}$ for Fiat-Shamir such that
 - $\mathbb{X}$ can be punctured for message m^* , that is:
 - (i) can sign all messages $m \neq m^*$
 - (ii) signature on m is hard to compute
 - $\mathbb{X}$ is compact and linear $\rightarrow$ efficient blind issuance
- **Statement $\mathbb{X}$:** inequality of encrypted messages

$C := C^* - \text{Enc}(\text{pk}, m; 0)$ does not encrypt 0

- **Puncturing:** encrypt m^* in C^*

Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

signer

$$\text{pk}, m$$

user

Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$vk = (C^*, \mathbf{D}), sk = d_1$$

$$pk, m$$

signer

user

C, π_m

$$C = \text{Enc}(pk, m; r)$$
$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$vk = (C^*, \mathbf{D}), sk = d_1$$

$$pk, m$$

signer

user

$$\mathbb{X} = C^* - C$$

C, π_m

$$C = \text{Enc}(pk, m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

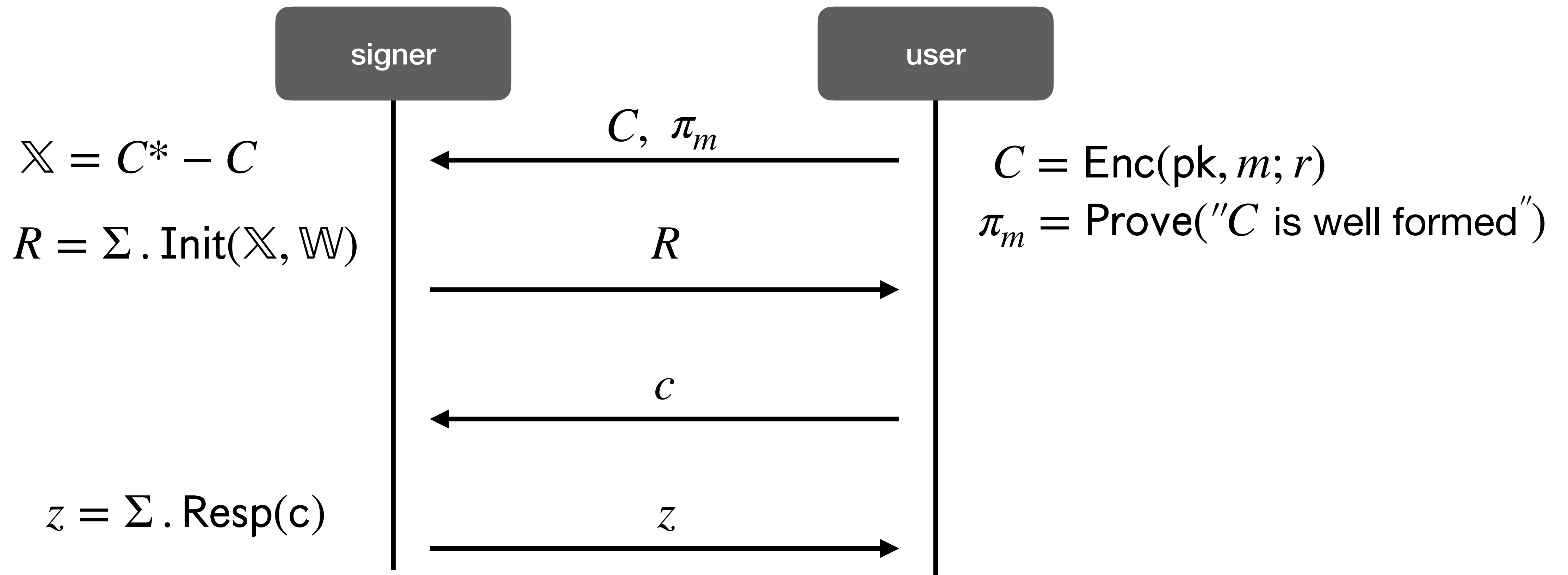
Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

$$\text{pk}, m$$



Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

$$\text{pk}, m$$

signer

user

$$\mathbb{X} = C^* - C$$

$$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$$

$$z = \Sigma . \text{Resp}(c)$$

$$C, \pi_m$$

$$R$$

$$c$$

$$z$$

$$C = \text{Enc}(\text{pk}, m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

compile proof $\pi = (R, c, z)$ via Fiat-Shamir

$$\sigma = \pi$$

Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$vk = (C^*, \mathbf{D}), sk = d_1$$

$$pk, m$$

signer

user

$$\mathbb{X} = C^* - C$$

$$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$$

$C^* - C$ does not encrypt 0
or
 $\mathbf{D}$ is DDH tuple

$$z = \Sigma . \text{Resp}(c)$$

$$C, \pi_m$$

$$R$$

$$c$$

$$z$$

$$C = \text{Enc}(pk, m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

compile proof $\pi = (R, c, z)$ via Fiat-Shamir

$$\sigma = \pi$$

Recap of [KR25]

Construction

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$vk = (C^*, \mathbf{D}), sk = d_1$$

$$pk, m$$

signer

user

$$\mathbb{X} = C^* - C$$

$$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$$

$C^* - C$ does not encrypt 0
or
 $\mathbf{D}$ is DDH tuple

$$z = \Sigma . \text{Resp}(c)$$

$$C, \pi_m$$

$$R$$

$$c$$

$$z$$

$$C = \text{Enc}(pk, m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

compile proof $\pi = (R, c, z)$ via Fiat-Shamir

$C^* - \text{Enc}(pk, m; 0)$ does not encrypt 0
or
 $\mathbf{D}$ is DDH tuple

Recap of [KR25]

Unforgeability

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

challenger

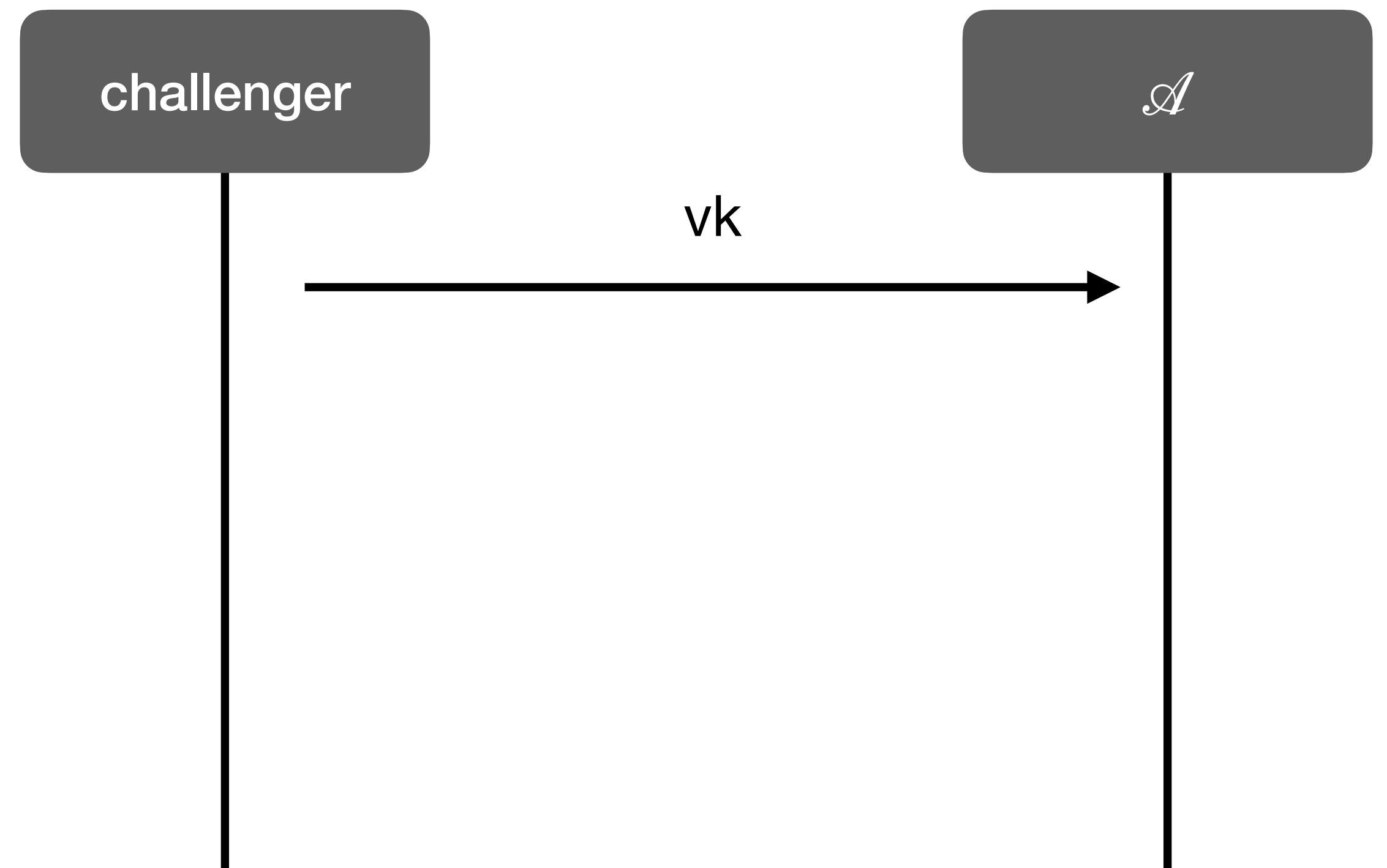
$\mathcal{A}$

Recap of [KR25]

Unforgeability

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

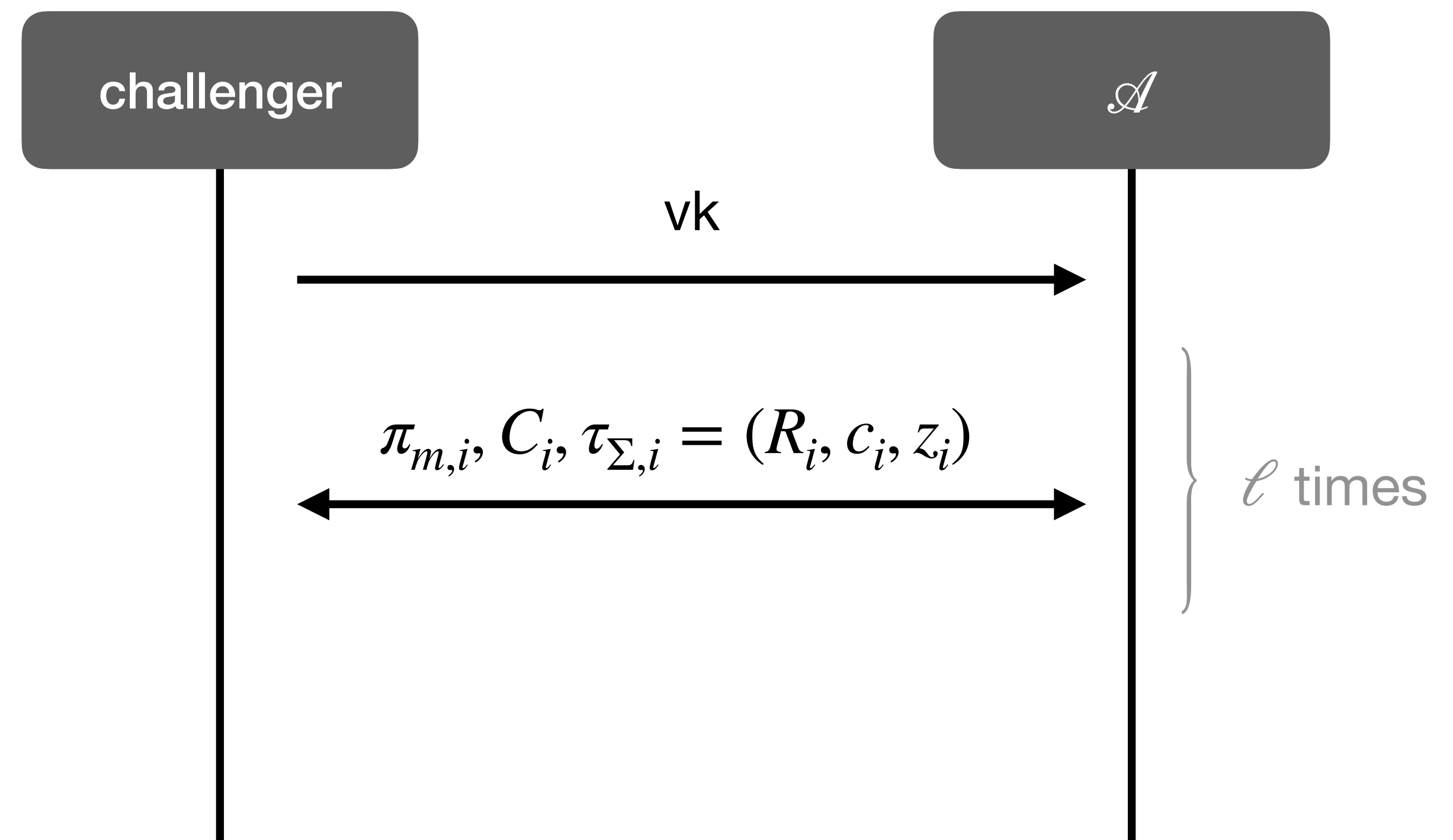


Recap of [KR25]

Unforgeability

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

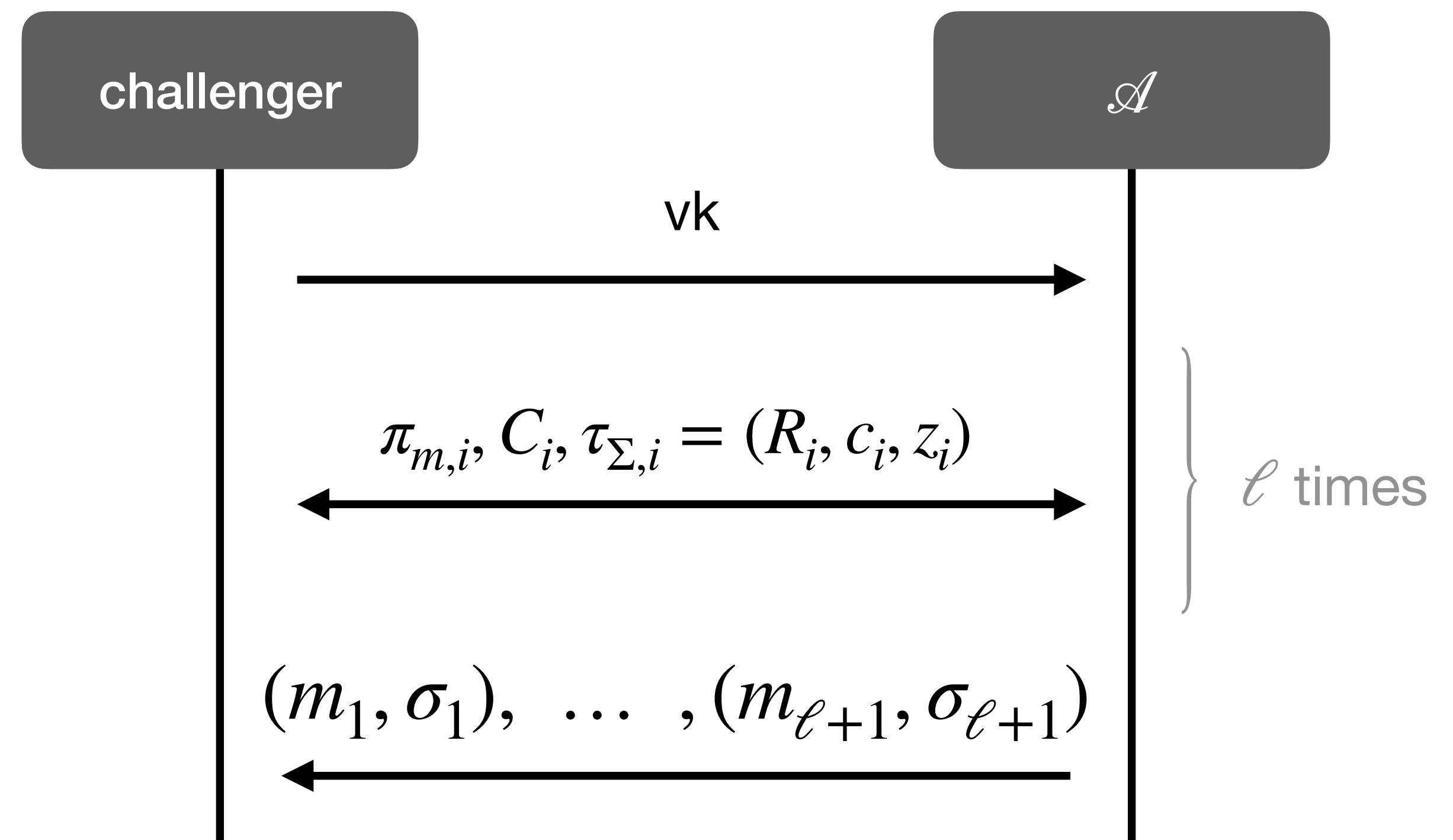


Recap of [KR25]

Unforgeability

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$

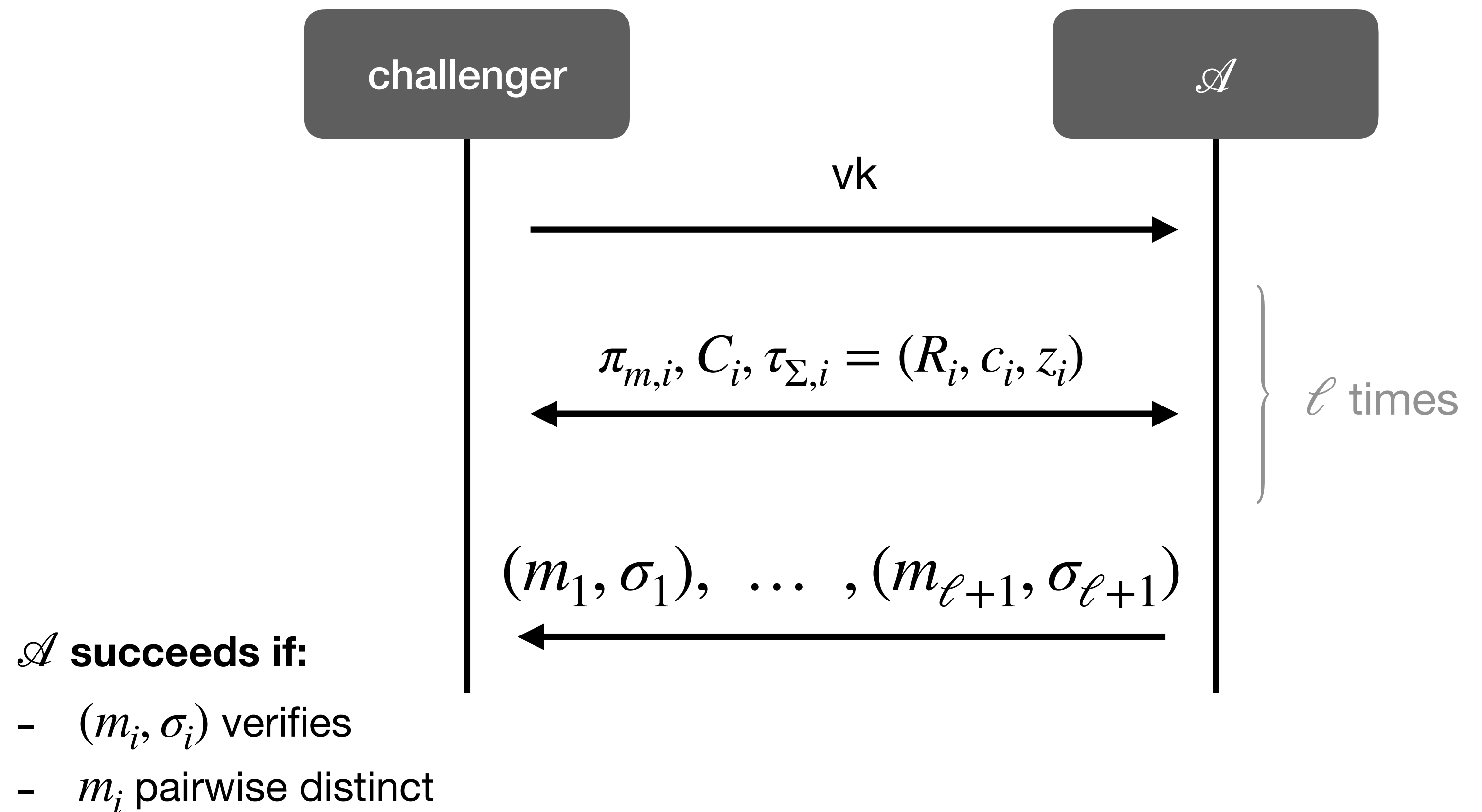


Recap of [KR25]

Unforgeability

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$



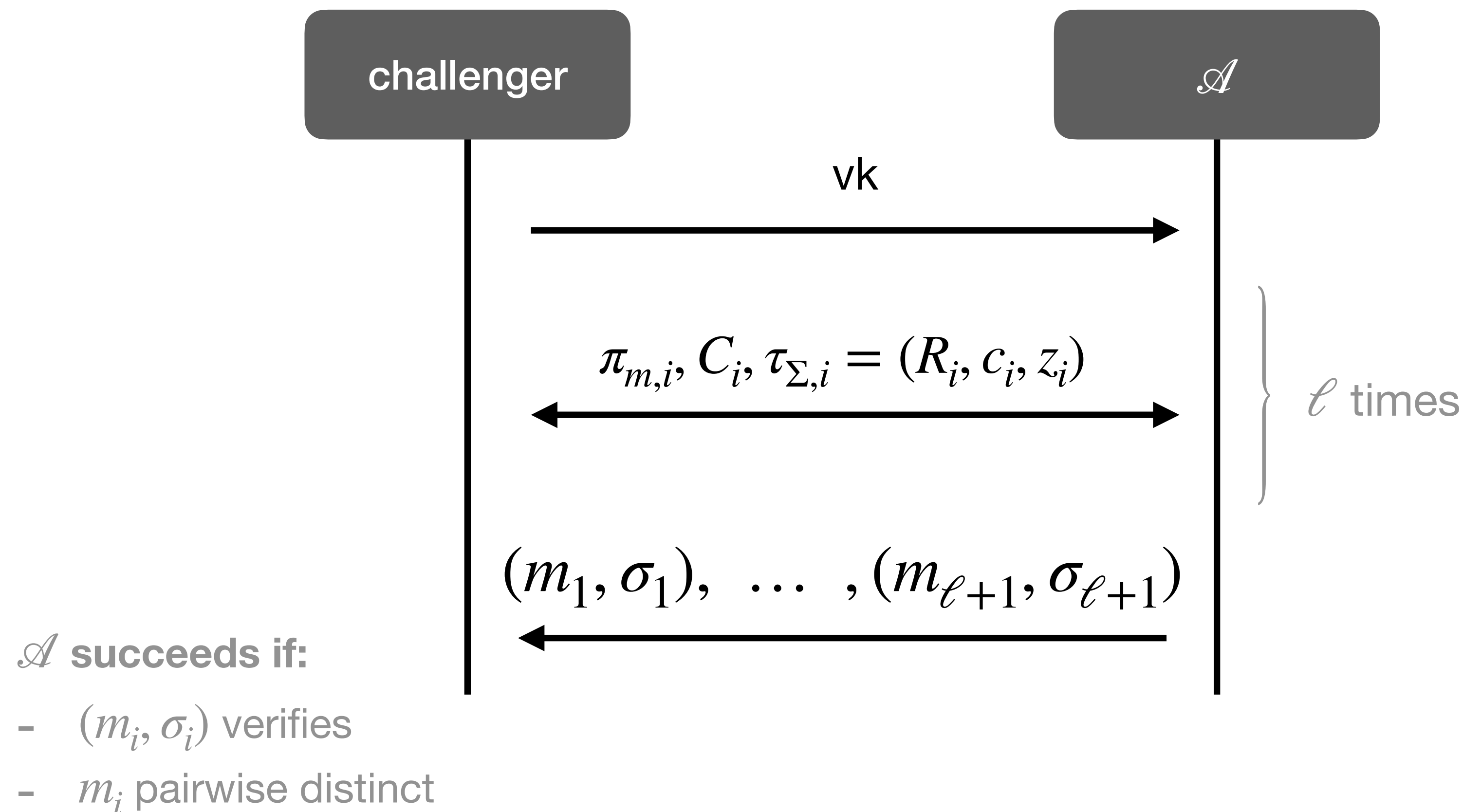
Recap of [KR25]

Unforgeability

- **Step 1:** puncture vk on some message m^*

$$\mathbf{D} = (D_1, D_2, D_3 = d_1 D_2)$$

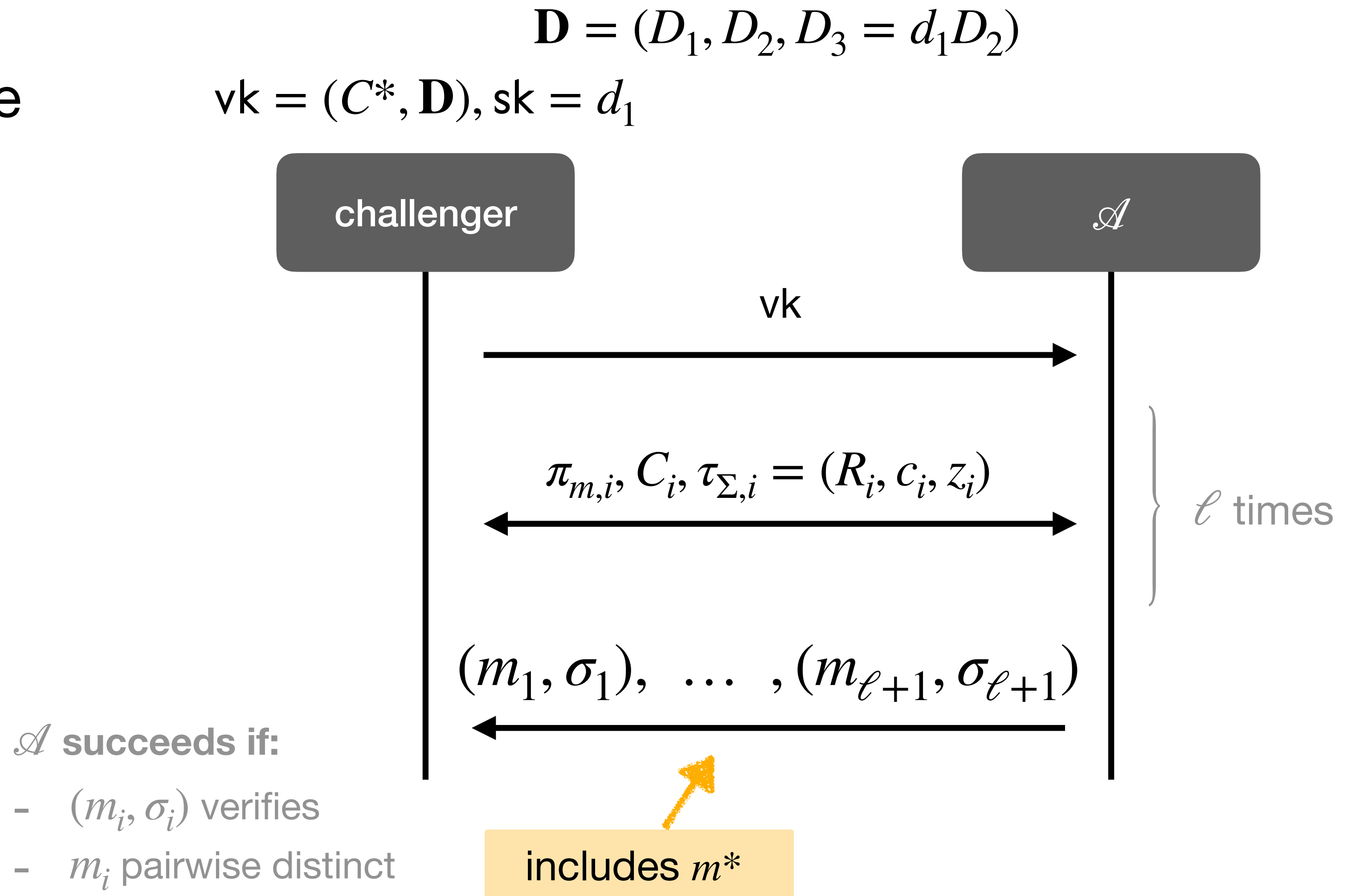
$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = d_1$$



Recap of [KR25]

Unforgeability

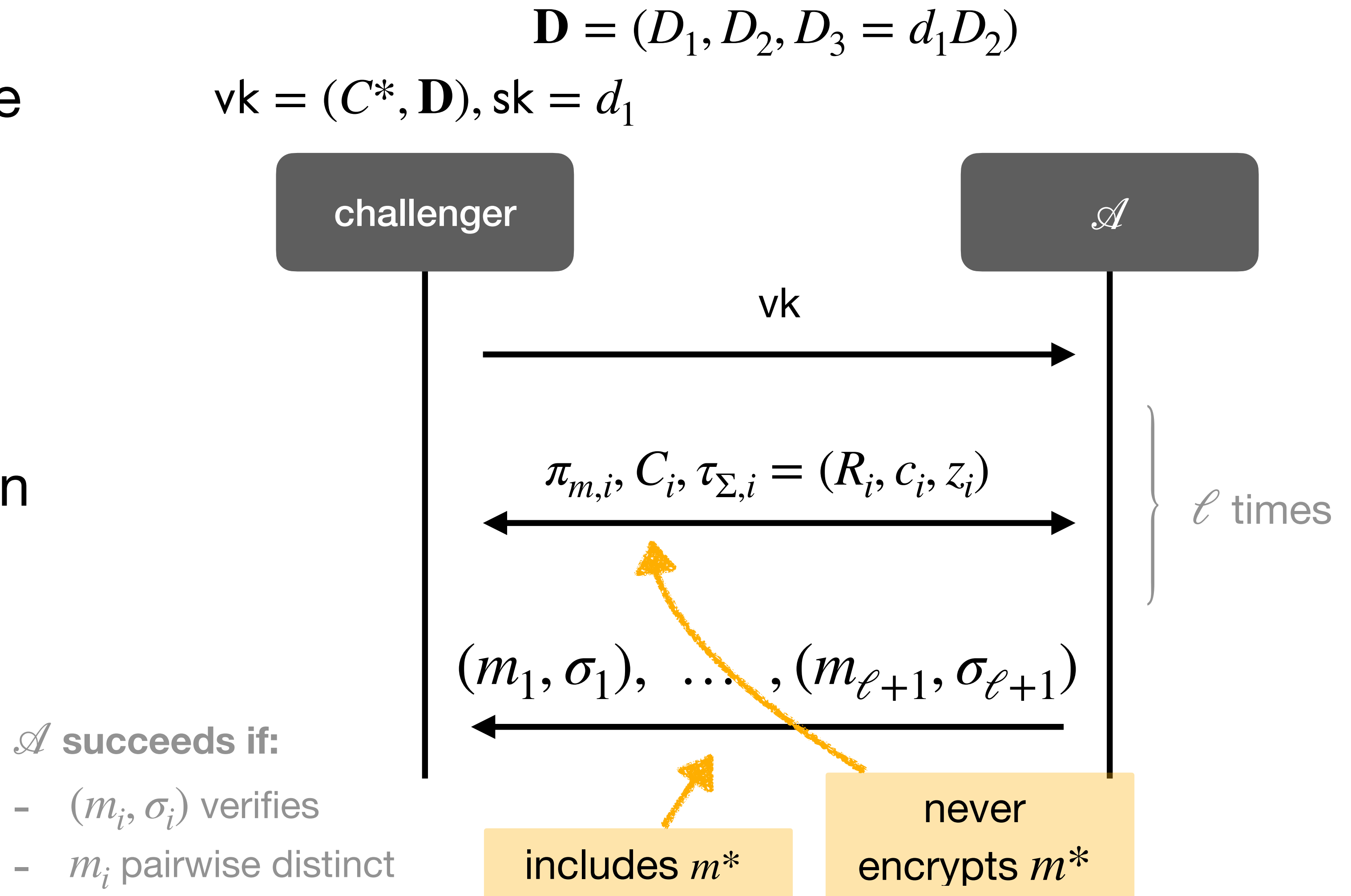
- **Step 1:** puncture vk on some message m^*
 - force adversary to provide forgery for m^*



Recap of [KR25]

Unforgeability

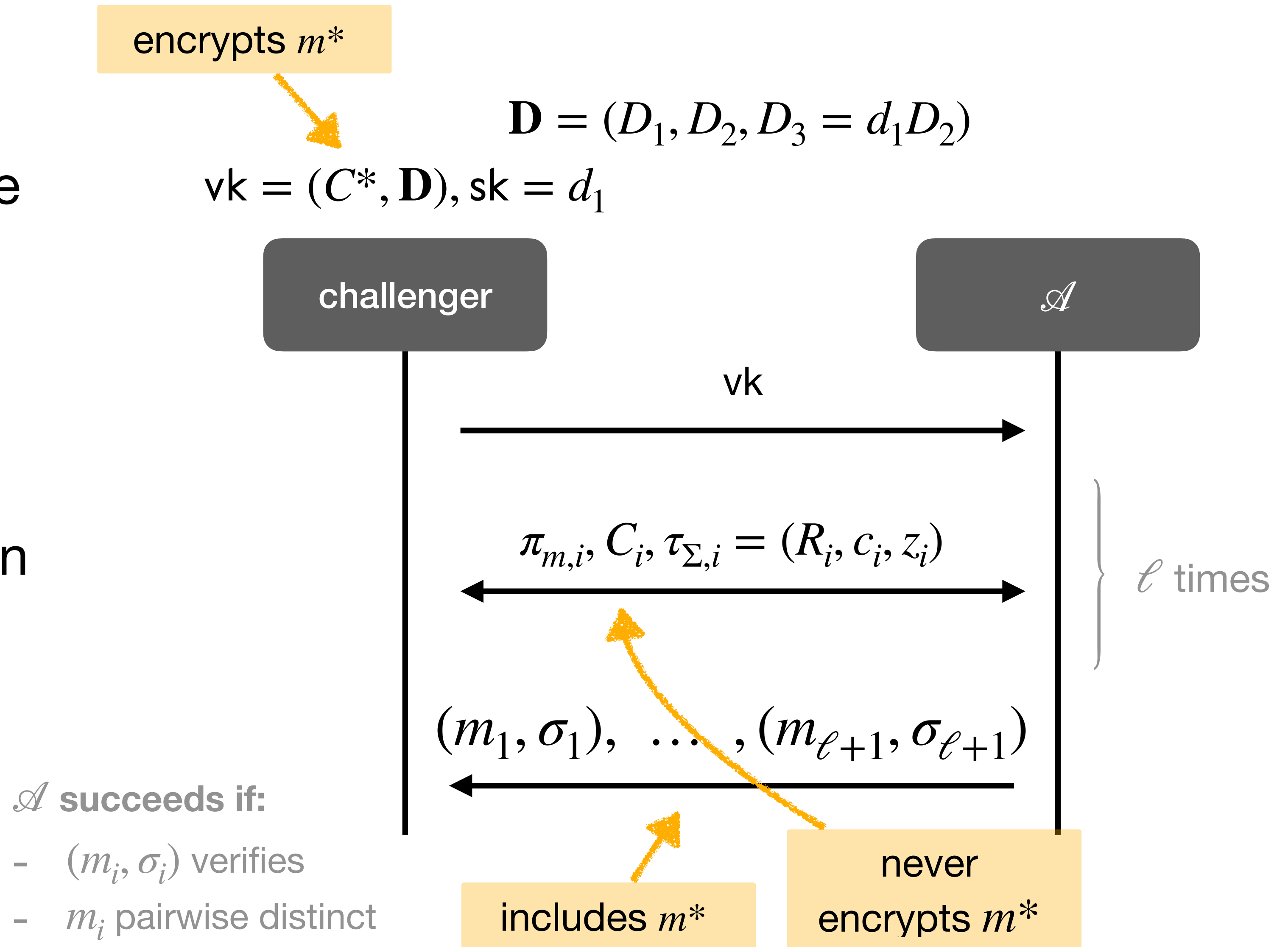
- **Step 1:** puncture vk on some message m^*
 - force adversary to provide forgery for m^*
 - never sign m^* in simulation



Recap of [KR25]

Unforgeability

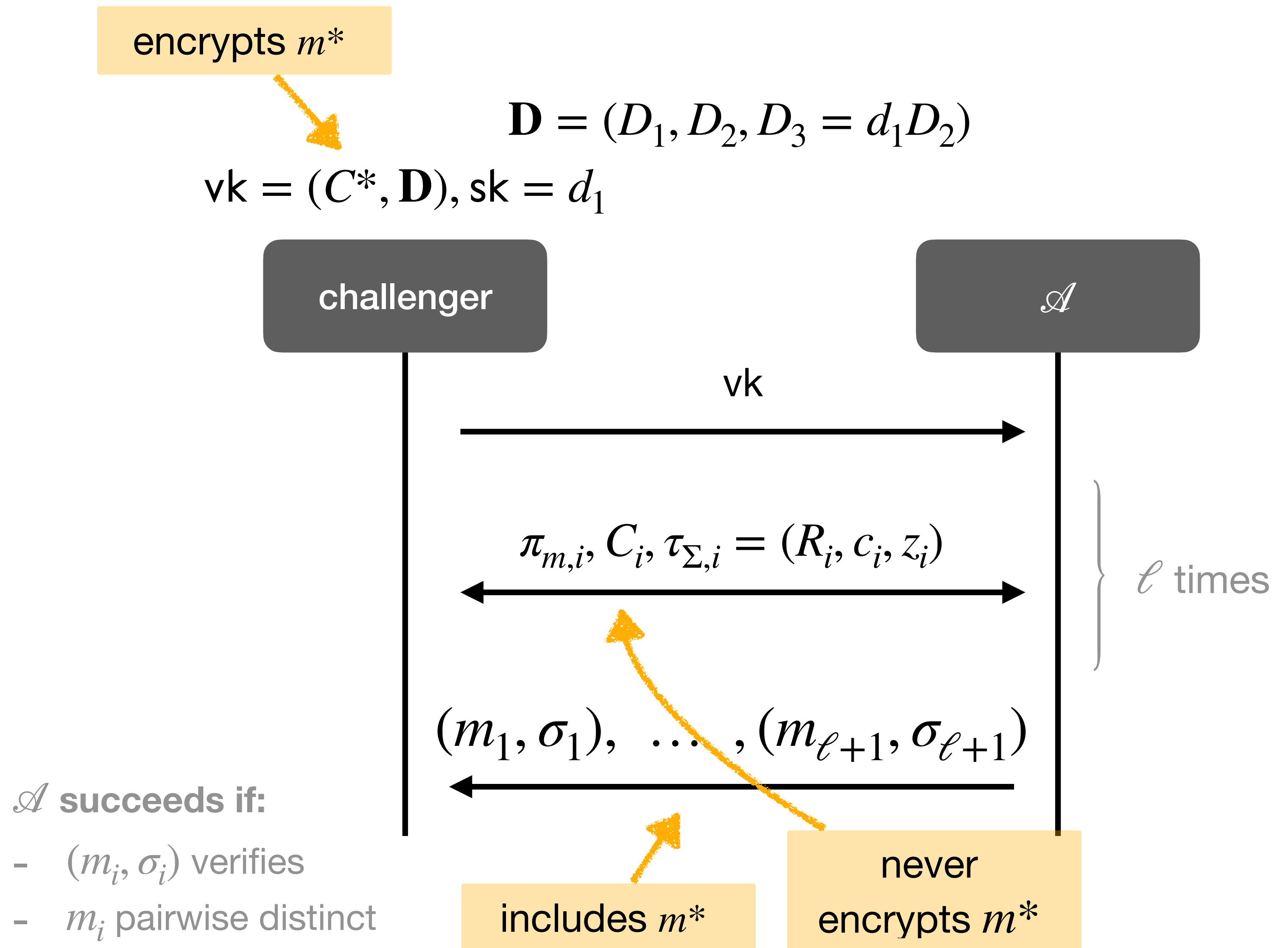
- **Step 1:** puncture vk on some message m^*
 - force adversary to provide forgery for m^*
 - never sign m^* in simulation
 - encrypt m^* in C^*



Recap of [KR25]

Unforgeability

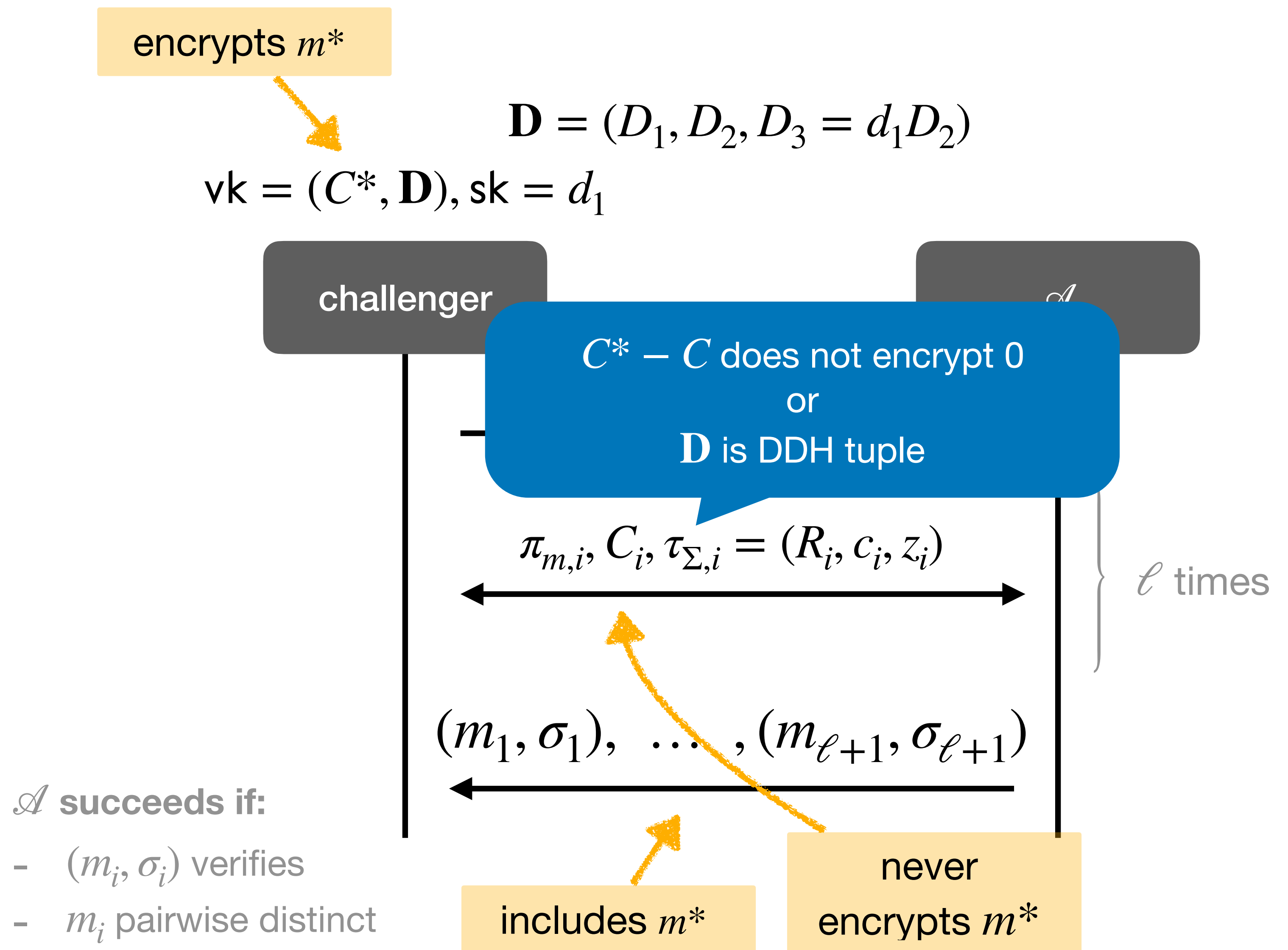
- **Step 2:** prove inequality branch honestly



Recap of [KR25]

Unforgeability

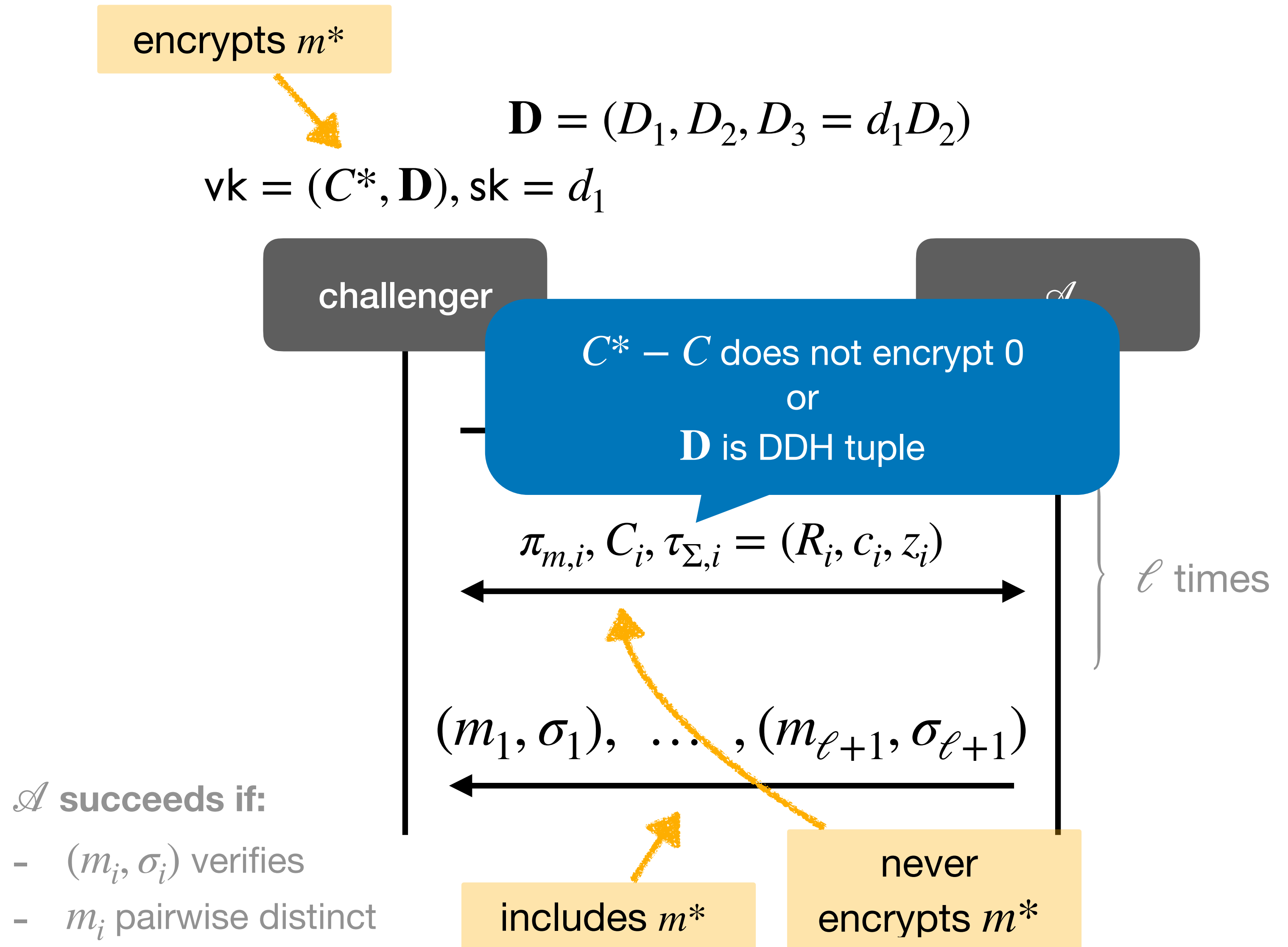
- **Step 2:** prove inequality branch honestly



Recap of [KR25]

Unforgeability

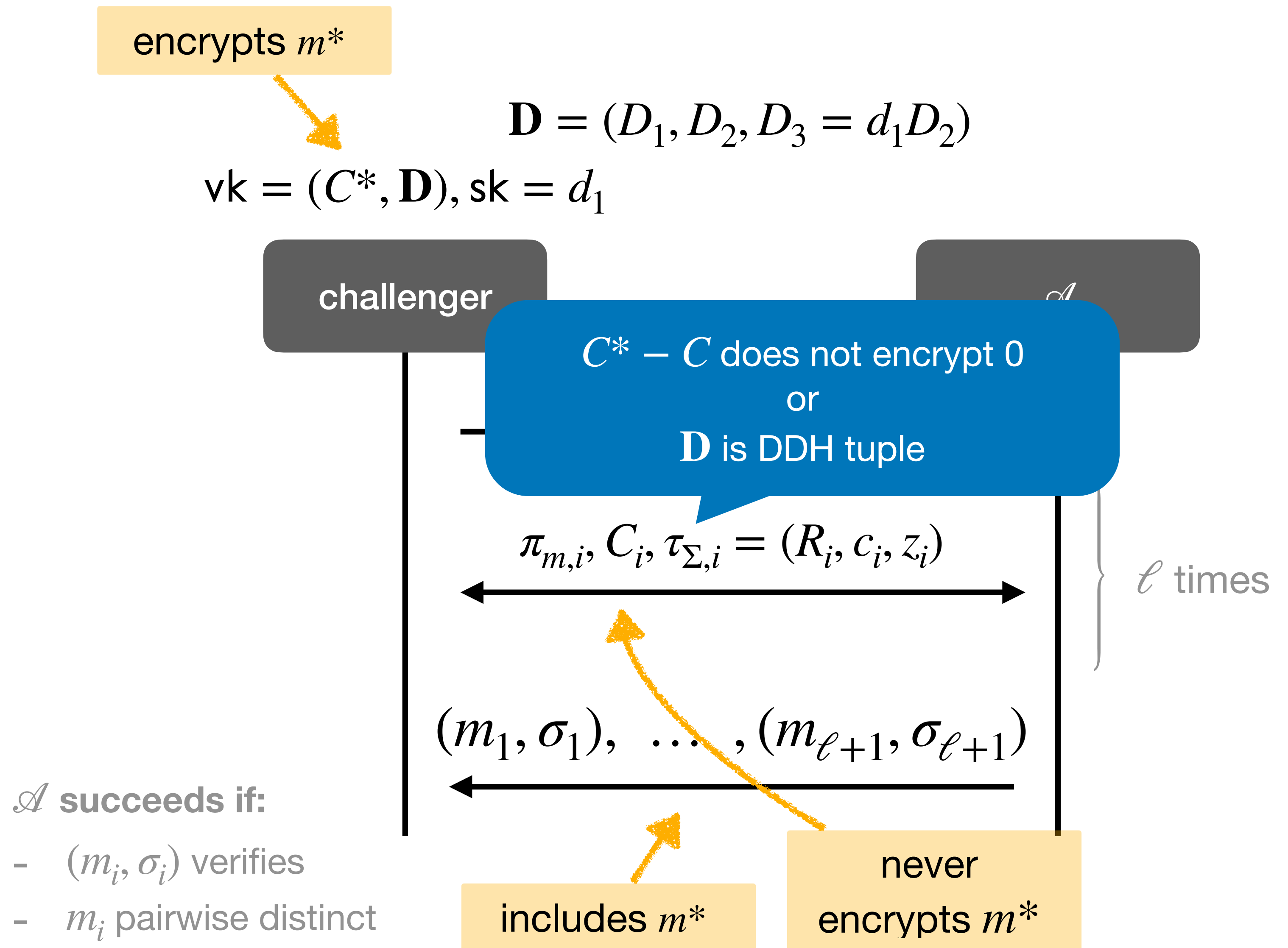
- **Step 2:** prove inequality branch honestly
 - extract to-be-signed m_i from proof $\pi_{m,i}$



Recap of [KR25]

Unforgeability

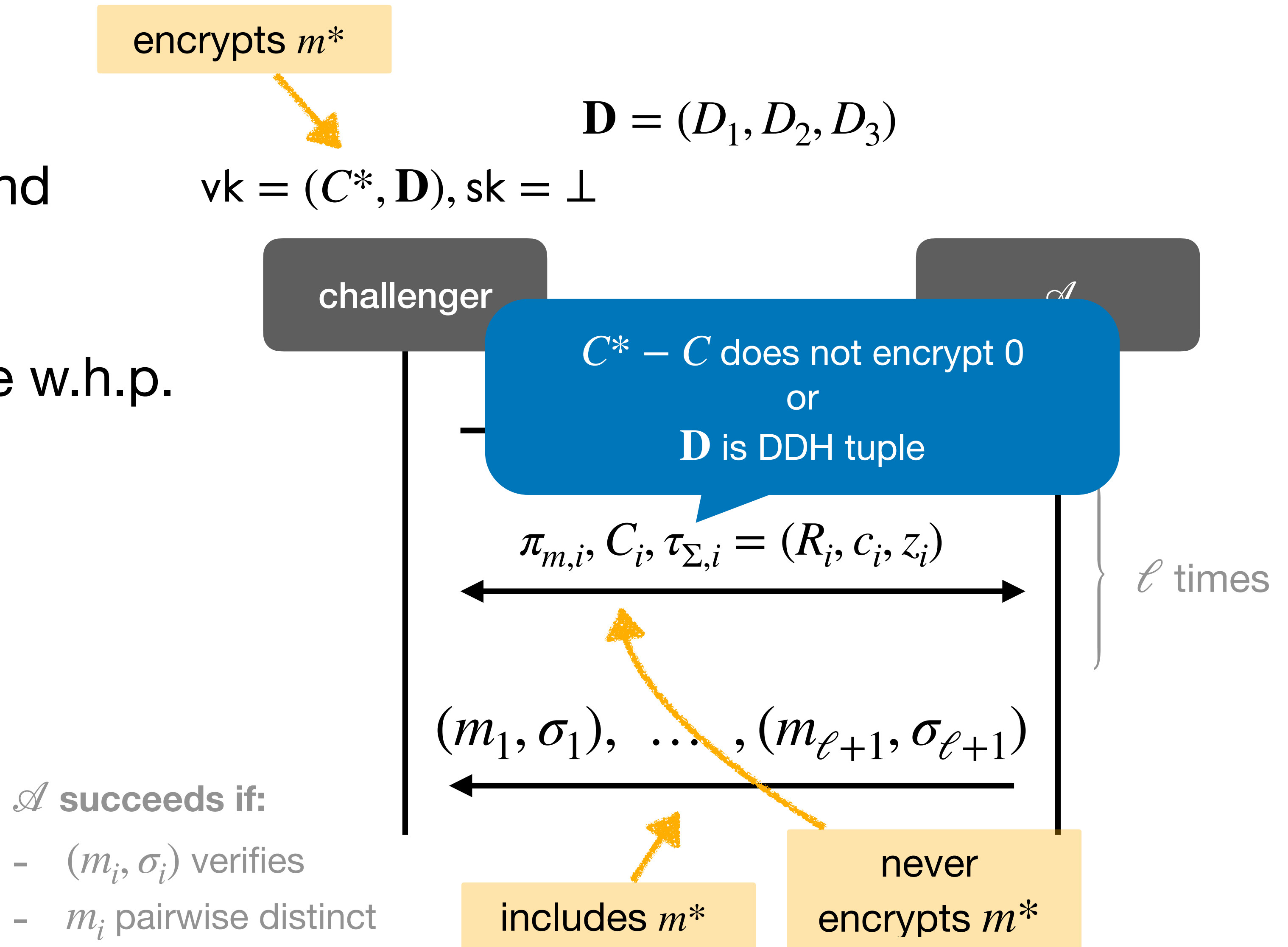
- **Step 2:** prove inequality branch honestly
 - extract to-be-signed m_i from proof $\pi_{m,i}$
 - compute witness to prove that $C^* - C$ does not encrypt 0, i.e., $m_i \neq m^*$



Recap of [KR25]

Unforgeability

- **Step 3:** sample random $\mathbf{D}$ and simulate $\mathbf{D}$ branch
 - note: $\mathbf{D}$ is not a DDH-tuple w.h.p.



Recap of [KR25]

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*

encrypts m^*

$$\mathbf{D} = (D_1, D_2, D_3)$$

$$\text{vk} = (C^*, \mathbf{D}), \text{sk} = \perp$$

challenger

$C^* - C$ does not encrypt 0
or
 $\mathbf{D}$ is DDH tuple

$$\pi_{m,i}, C_i, \tau_{\Sigma,i} = (R_i, c_i, z_i)$$

ℓ times

$$(m_1, \sigma_1), \dots, (m_{\ell+1}, \sigma_{\ell+1})$$

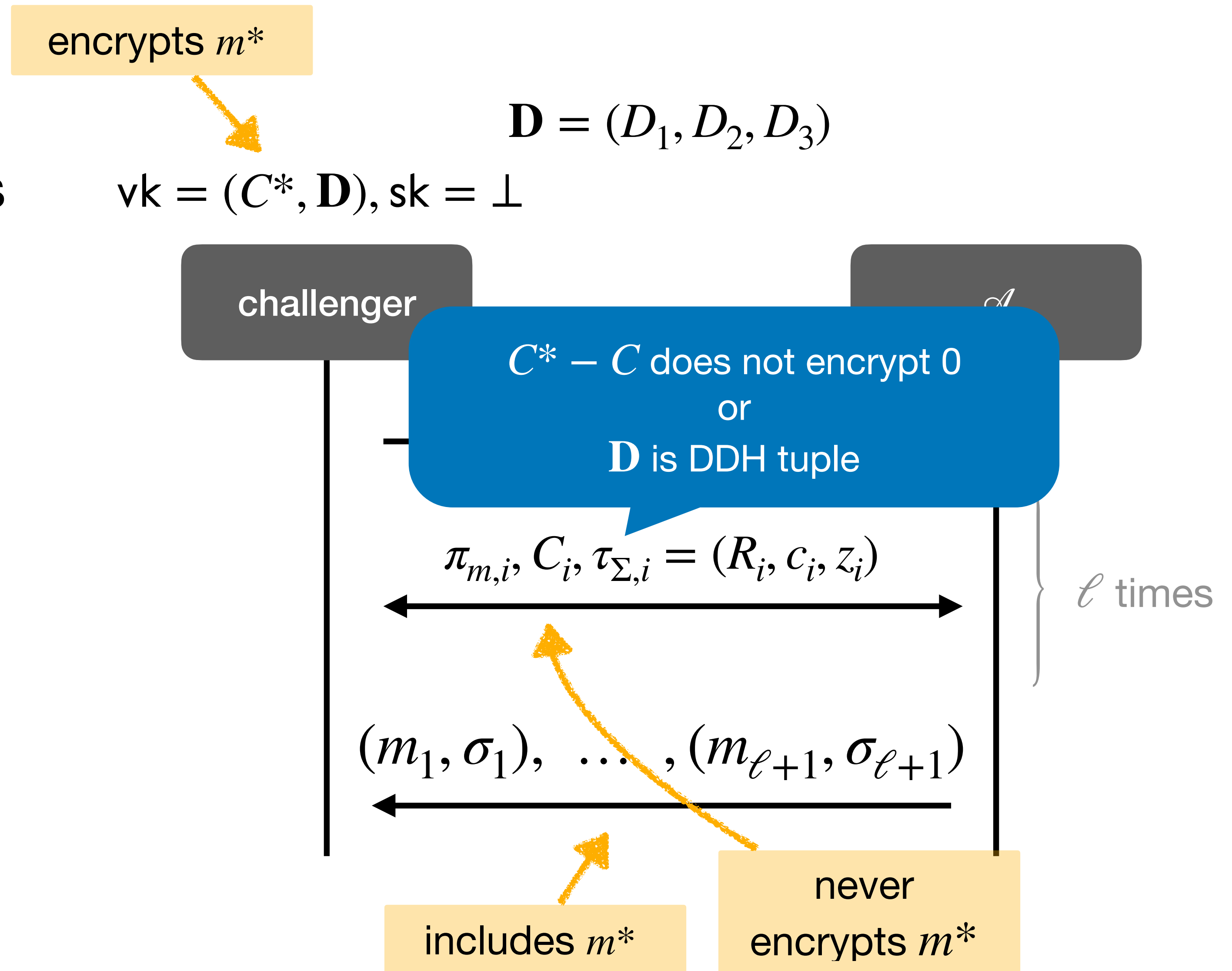
includes m^*

never
encrypts m^*

Recap of [KR25]

Unforgeability

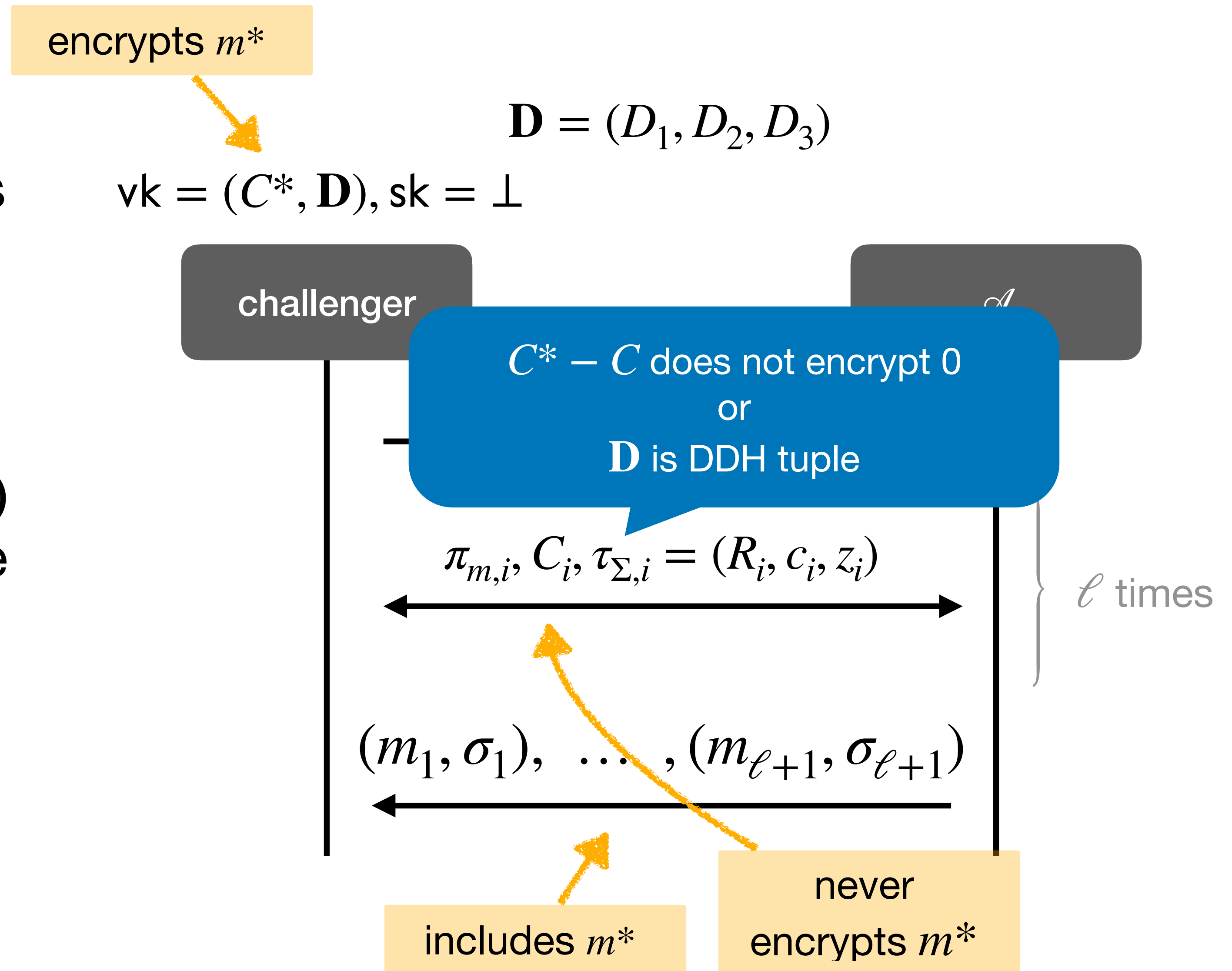
- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that



Recap of [KR25]

Unforgeability

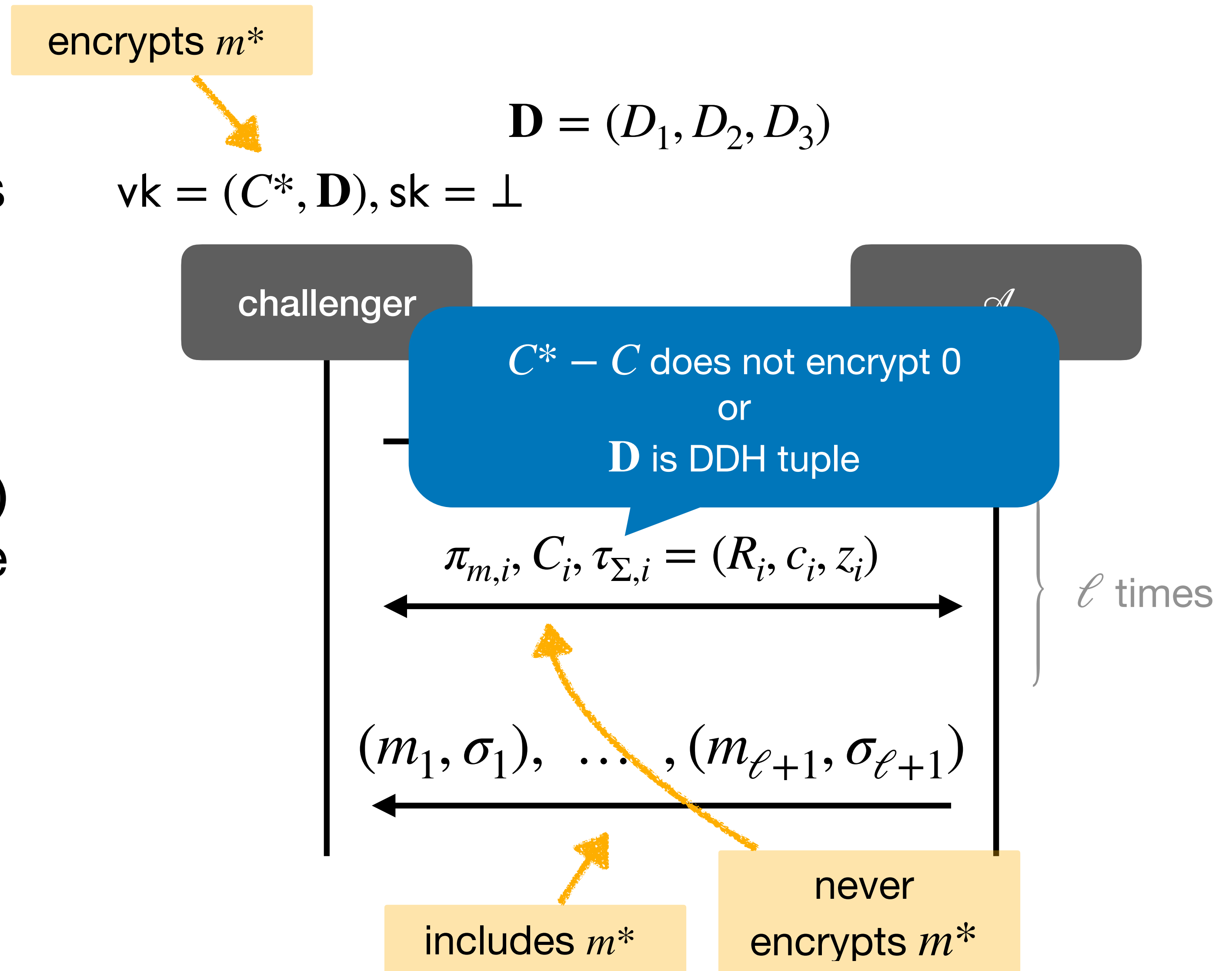
- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Enc}(\text{pk}, m^*; 0)$ encrypts non-zero message



Recap of [KR25]

Unforgeability

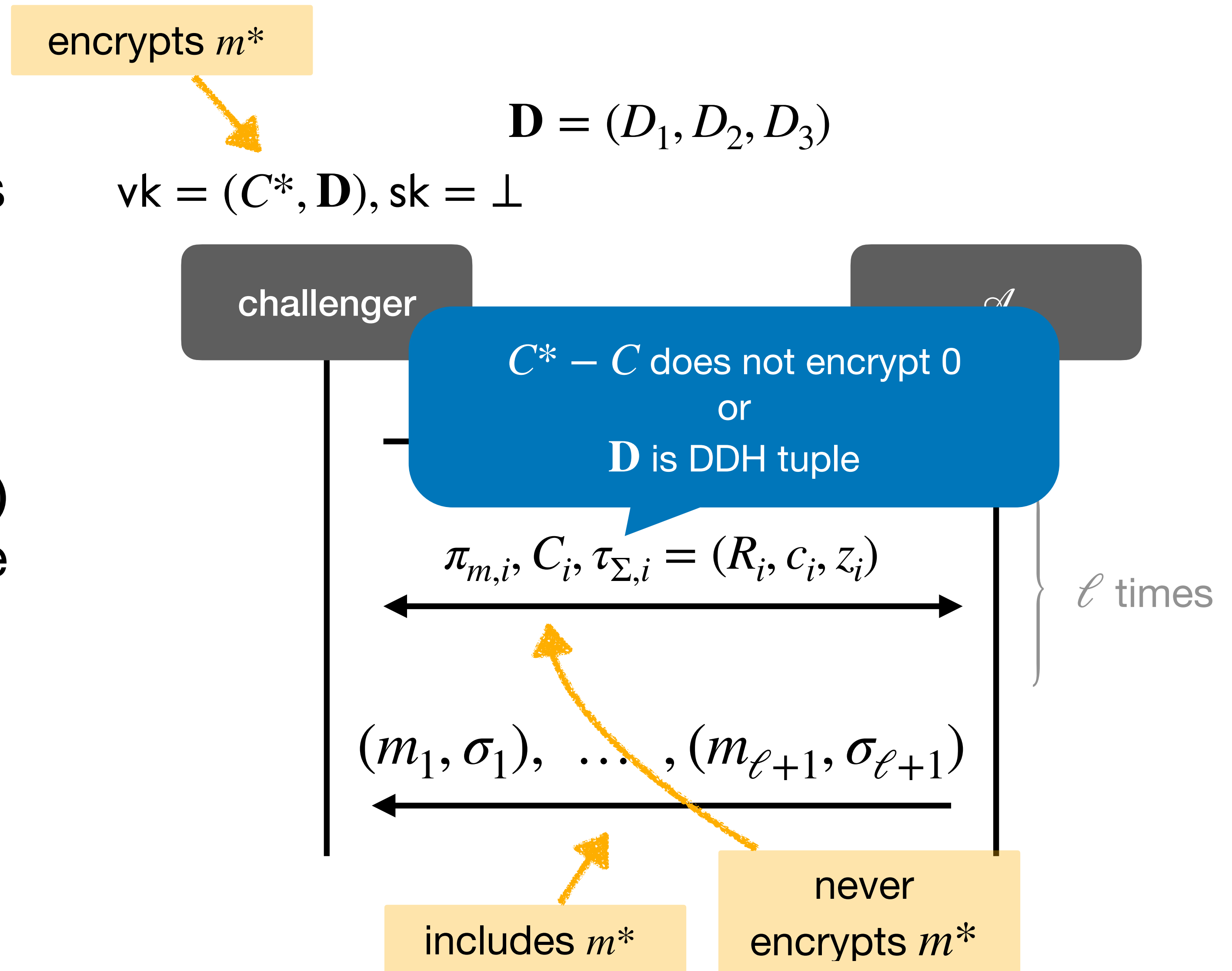
- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* = \text{Enc}(\text{pk}, m^*; 0)$ encrypts non-zero message
 - or $\mathbf{D}$ is a DDH tuple



Recap of [KR25]

Unforgeability

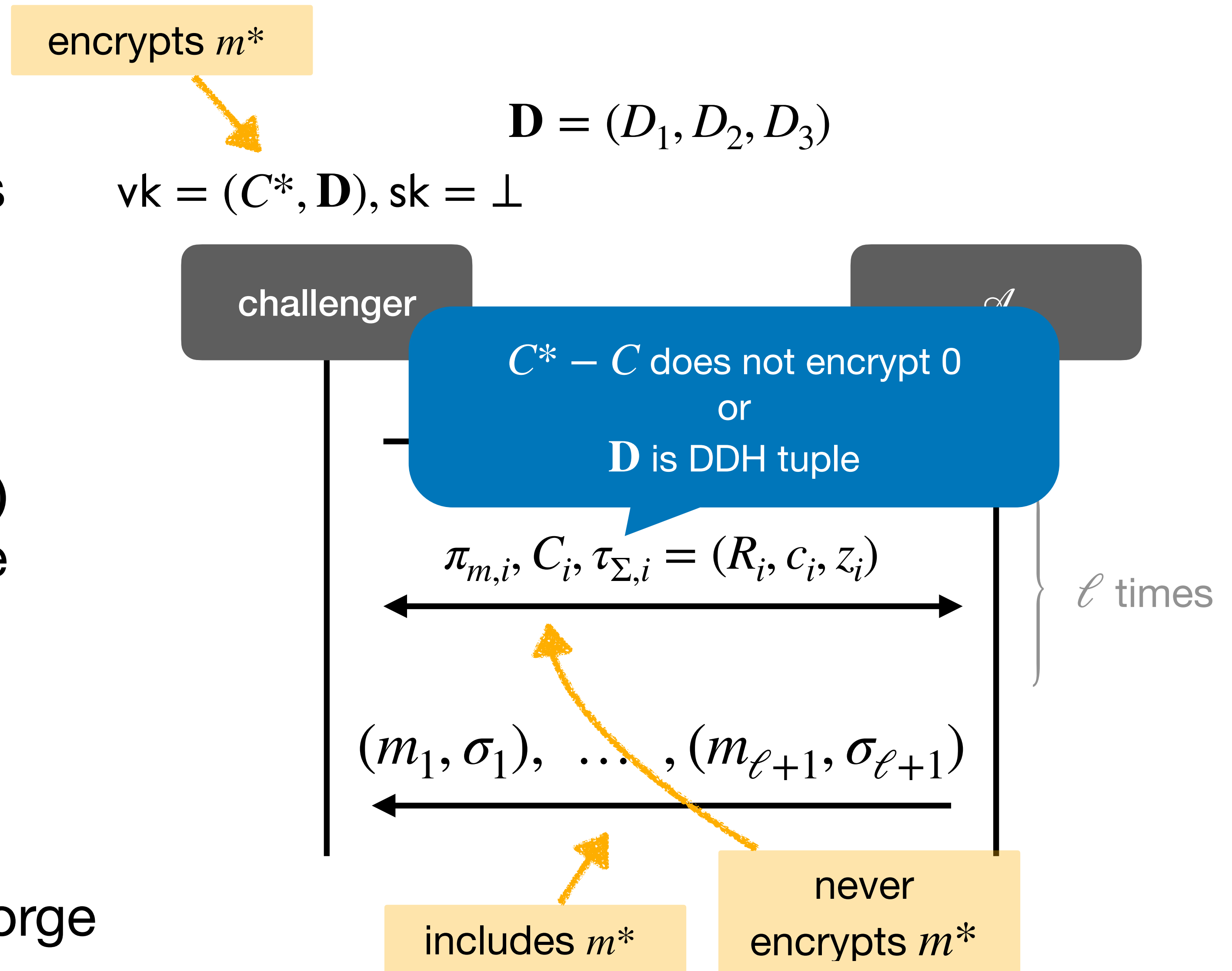
- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* = \text{Enc}(\text{pk}, m^*; 0)$ encrypts non-zero message
 - or $\mathbf{D}$ is a DDH tuple
 - both claims are *false*



Recap of [KR25]

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Enc}(\text{pk}, m^*; 0)$ encrypts non-zero message
 - or $\mathbf{D}$ is a DDH tuple
 - both claims are *false*
 - *soundness* $\rightarrow$ impossible to forge



Our Approach

Our Approach [KLR26]

Blind Signatures from DL

Our Approach [KLR26]

Blind Signatures from DL

- **Caveat of [KR25]:** relies on DDH for encryption

Our Approach [KLR26]

Blind Signatures from DL

- **Caveat of [KR25]:** relies on DDH for encryption
- **Idea:** replace encryption with commitments

Our Approach [KLR26]

Blind Signatures from DL

- **Caveat of [KR25]:** relies on DDH for encryption
- **Idea:** replace encryption with commitments
- **Statement $\times$:** inequality of committed messages

Our Approach [KLR26]

Blind Signatures from DL

- **Caveat of [KR25]:** relies on DDH for encryption
- **Idea:** replace encryption with commitments
- **Statement $\mathbb{X}$:** inequality of committed messages

$C := C^* - \text{Com}(m; 0)$ does not commit to 0

Our Approach [KLR26]

Blind Signatures from DL

- **Caveat of [KR25]:** relies on DDH for encryption
- **Idea:** replace encryption with commitments
- **Statement $\mathbb{X}$:** inequality of committed messages

$C := C^* - \text{Com}(m; 0)$ does not commit to 0

→ hides message m statistically, fixes m under DL assumption

Our Approach [KLR26]

Construction

$$D = dG$$

$$vk = (C^*, D), sk = d$$

signer

$$vk, m$$

user

Our Approach [KLR26]

Construction

$$D = dG$$

$$vk = (C^*, D), sk = d$$

signer

$$vk, m$$

user

$$C, \pi_m$$

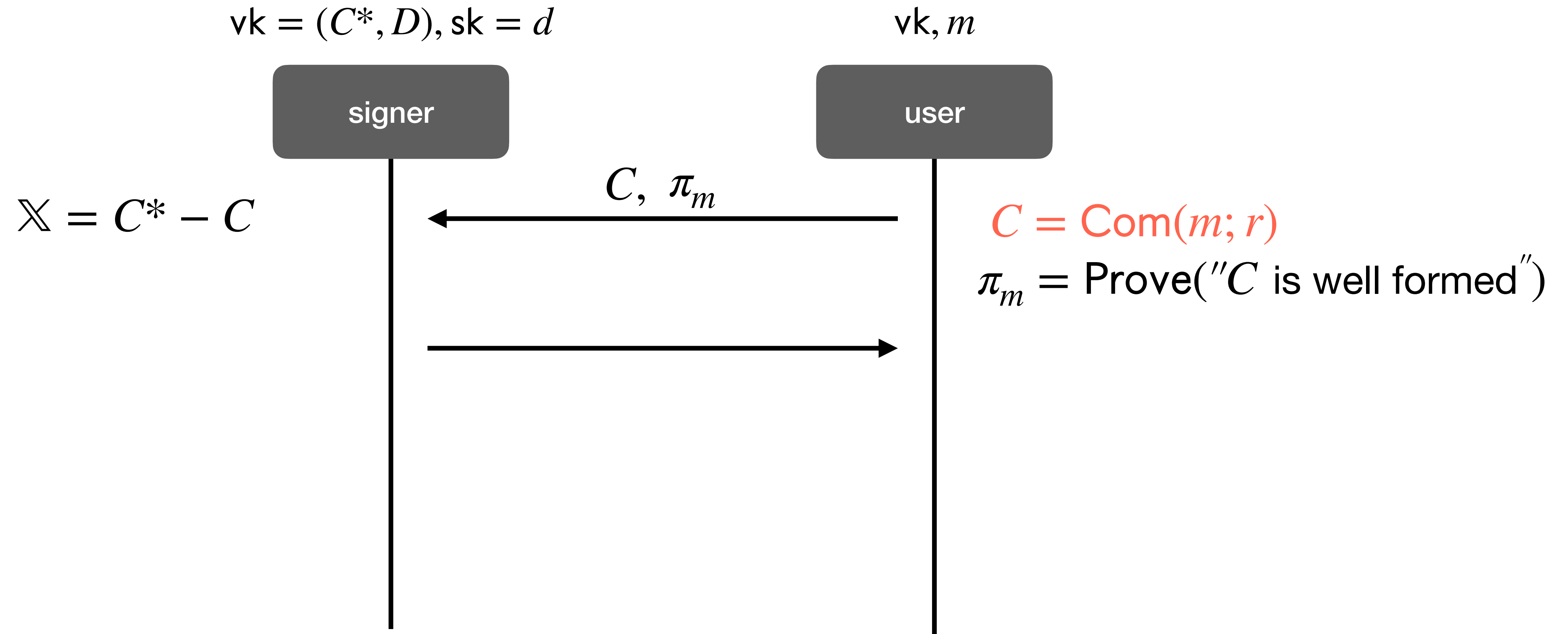
$$C = \text{Com}(m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed"})$$

Our Approach [KLR26]

Construction

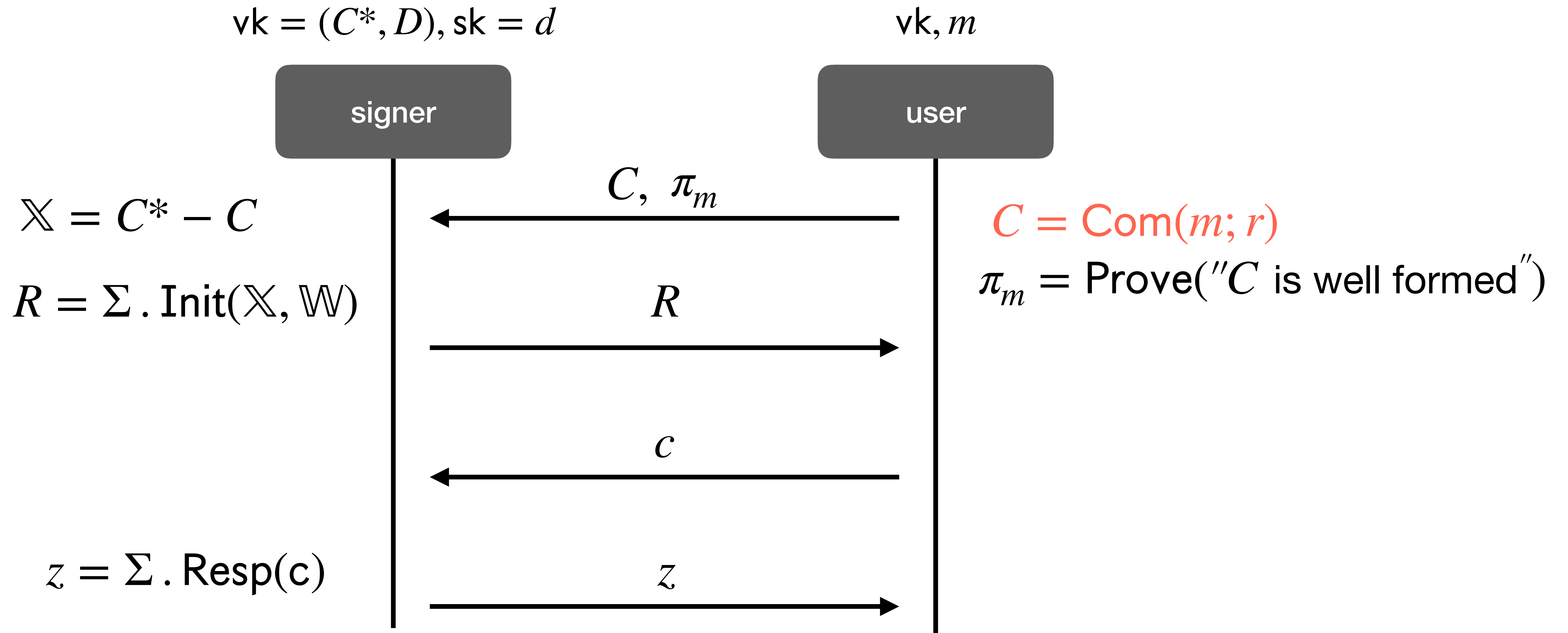
$$D = dG$$



Our Approach [KLR26]

Construction

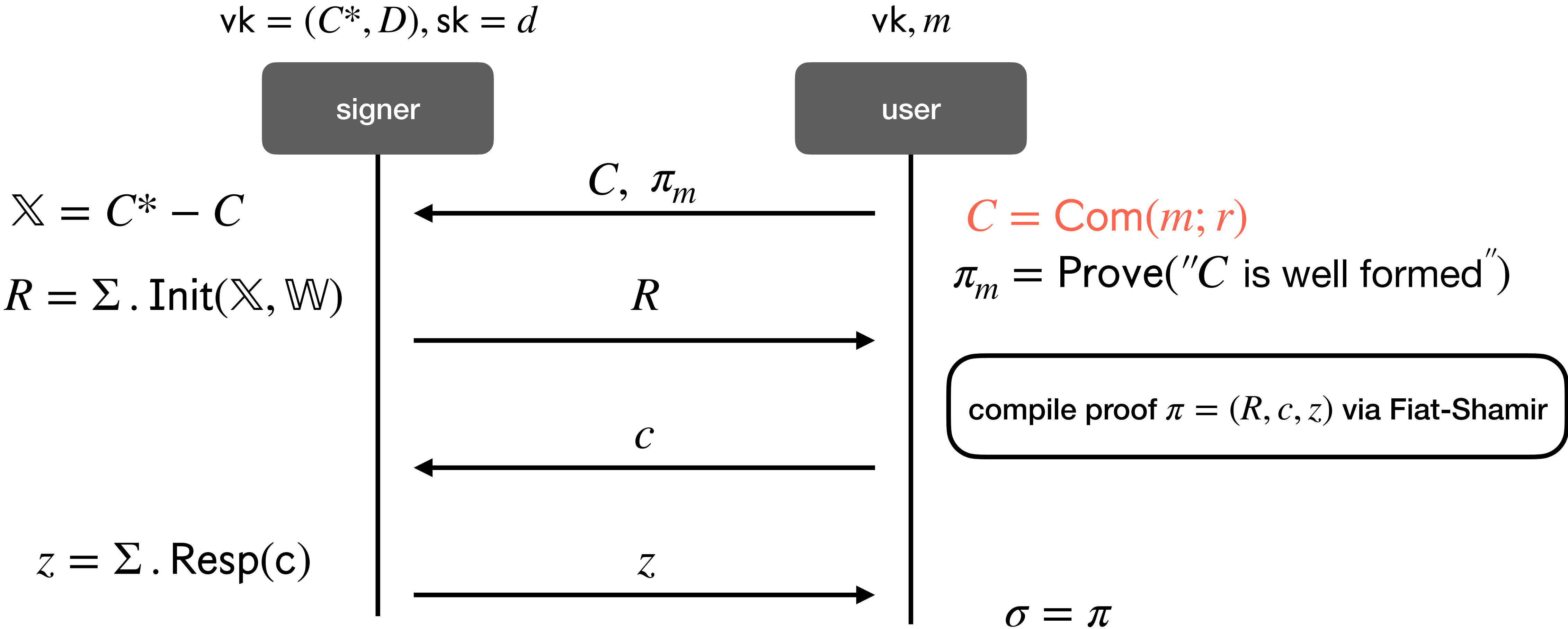
$$D = dG$$



Our Approach [KLR26]

Construction

$$D = dG$$



Our Approach [KLR26]

Construction

$$D = dG$$

$$vk = (C^*, D), sk = d$$

$$vk, m$$

signer

user

$$\mathbb{X} = C^* - C$$

$$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$$

$C^* - C$ does not **commit** to 0
or
 $D = dG$

$$z = \Sigma . \text{Resp}(c)$$

C, π_m

R

c

z

$$C = \text{Com}(m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

compile proof $\pi = (R, c, z)$ via Fiat-Shamir

$$\sigma = \pi$$

Our Approach [KLR26]

Construction

$$D = dG$$

$$vk = (C^*, D), sk = d$$

$$vk, m$$

signer

user

$$\mathbb{X} = C^* - C$$

$$R = \Sigma . \text{Init}(\mathbb{X}, \mathbb{W})$$

$C^* - C$ does not **commit** to 0
or

$$D = dG$$

$$z = \Sigma . \text{Resp}(c)$$

$$C, \pi_m$$

$$R$$

$$c$$

$$z$$

$$C = \text{Com}(m; r)$$

$$\pi_m = \text{Prove}("C \text{ is well formed}")$$

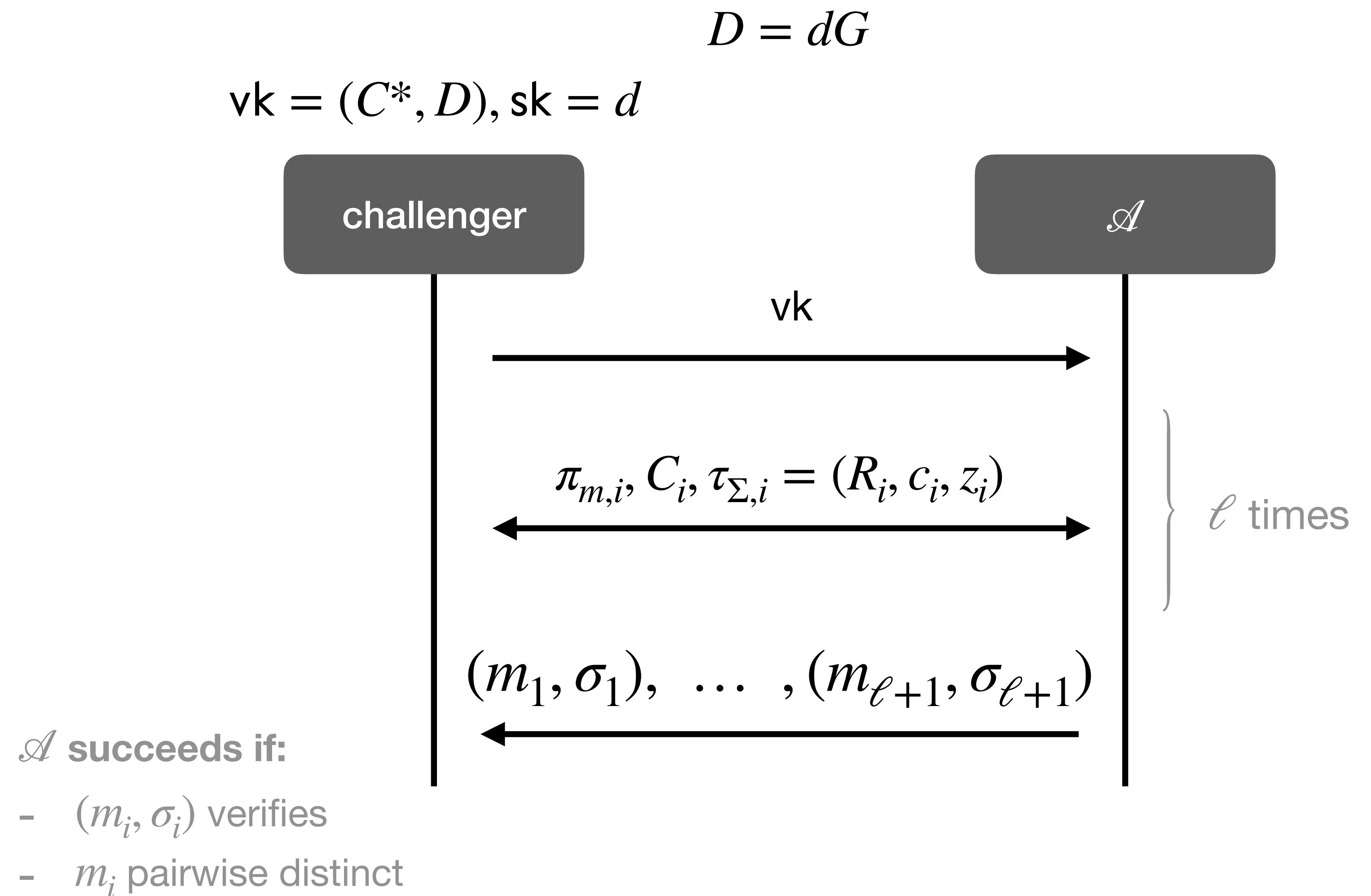
compile proof $\pi = (R, c, z)$ via Fiat-Shamir

$C^* - \text{Com}(m; 0)$ does not **commit** to 0
or

$$D = dG$$

Our Approach [KLR26]

Unforgeability



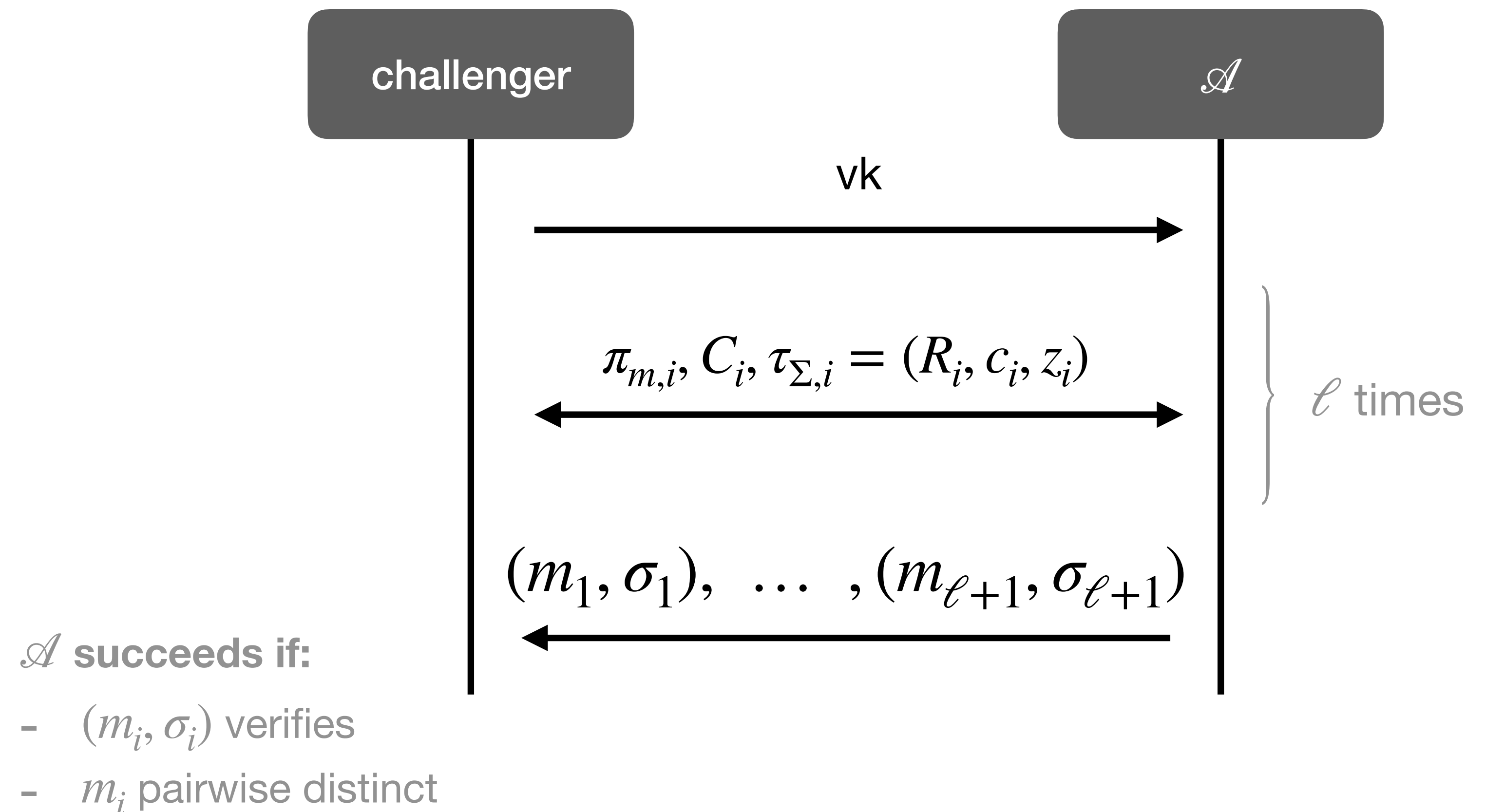
Our Approach [KLR26]

Unforgeability

- **Step 1:** puncture vk on some message m^*

$$D = dG$$

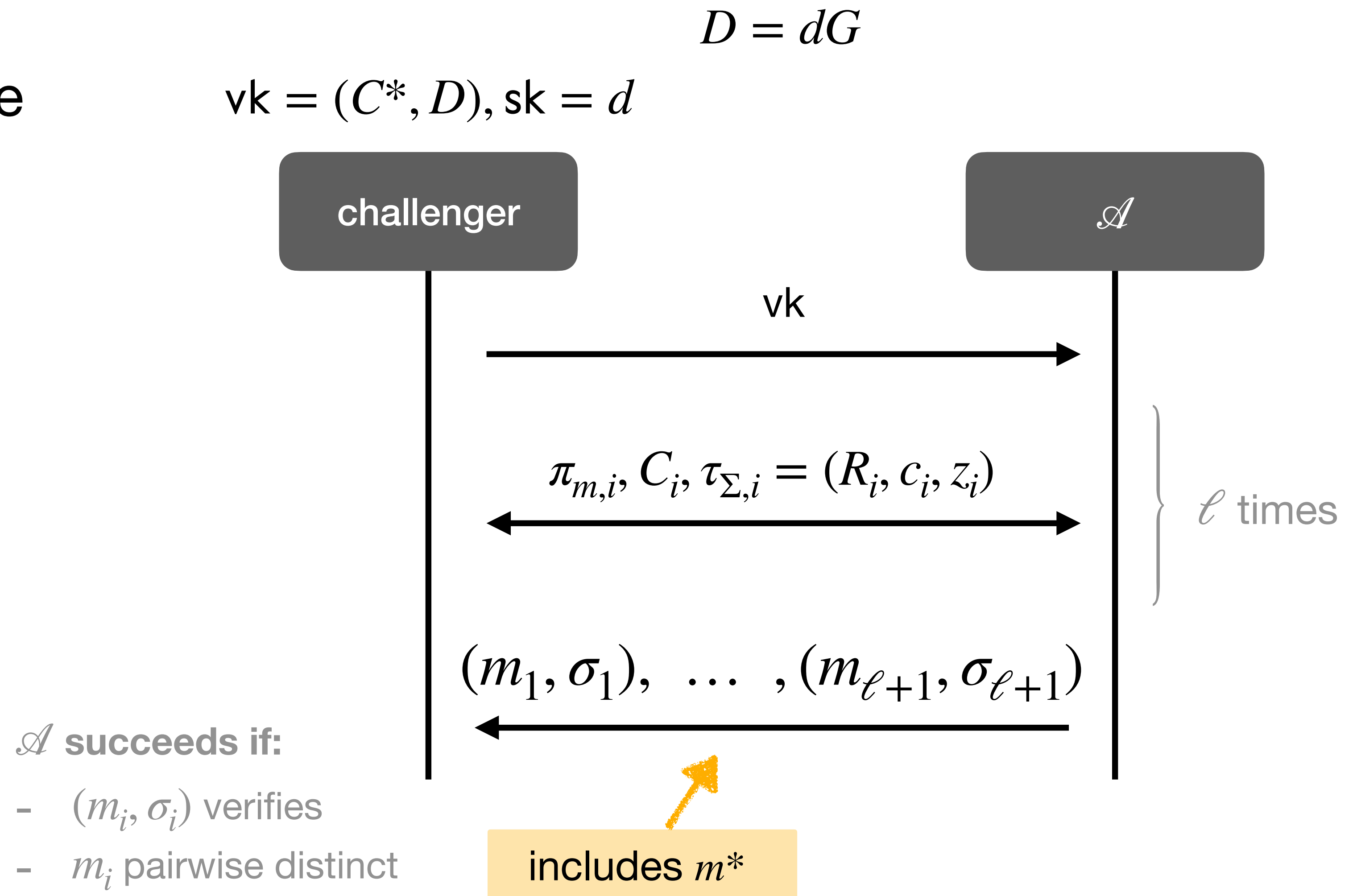
$$vk = (C^*, D), sk = d$$



Our Approach [KLR26]

Unforgeability

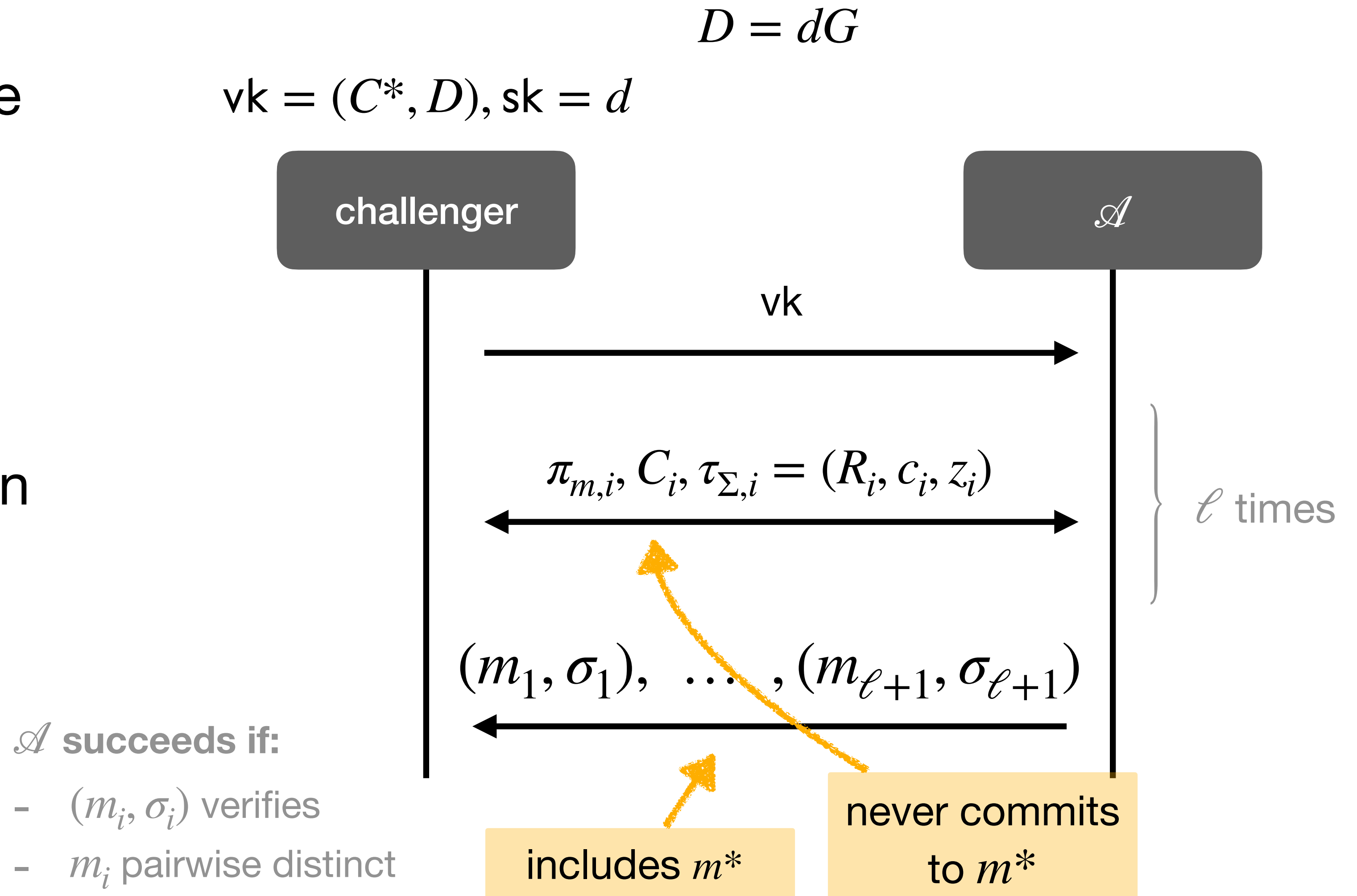
- **Step 1:** puncture vk on some message m^*
 - force adversary to provide forgery for m^*



Our Approach [KLR26]

Unforgeability

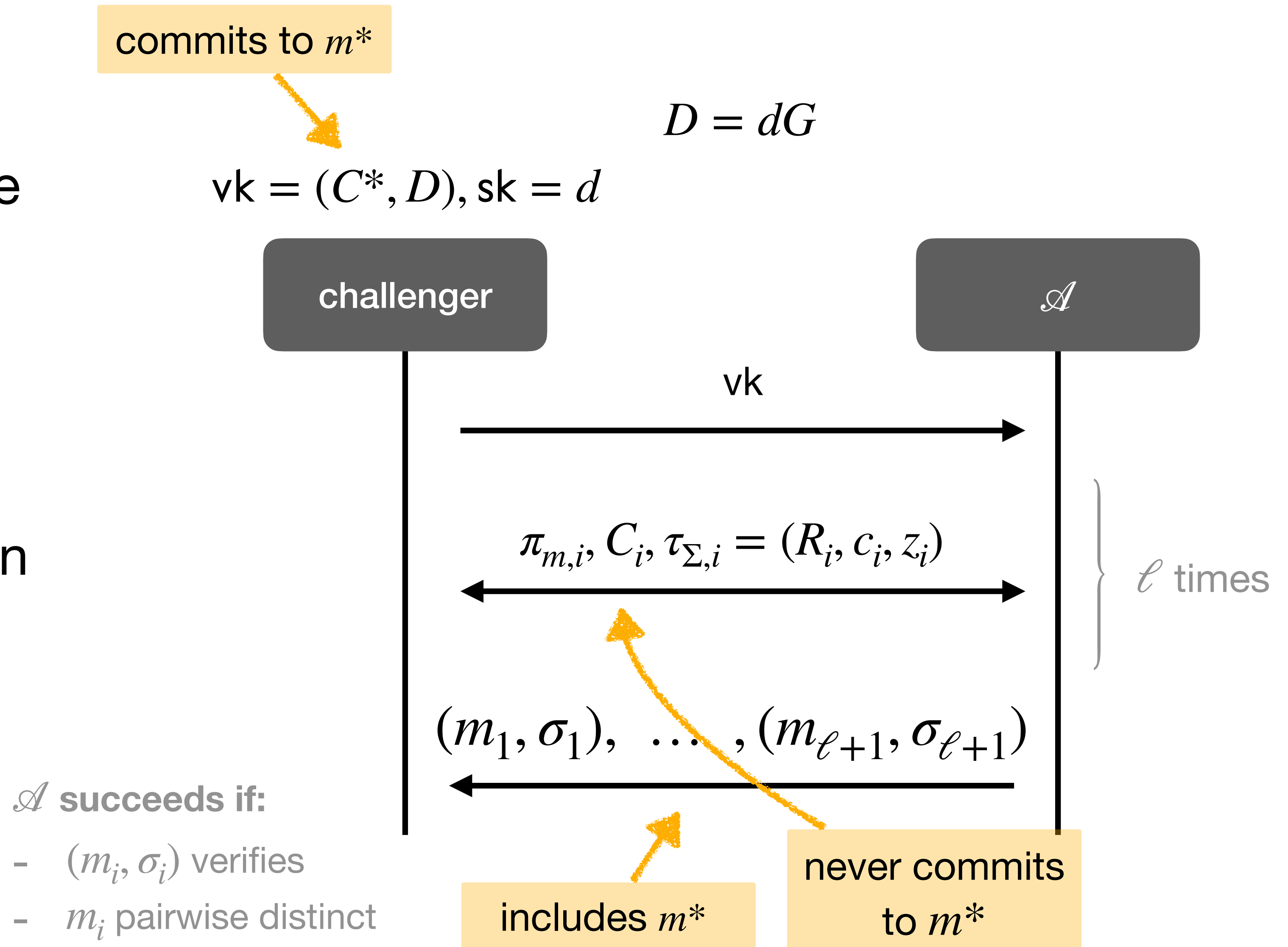
- **Step 1:** puncture vk on some message m^*
 - force adversary to provide forgery for m^*
 - never sign m^* in simulation



Our Approach [KLR26]

Unforgeability

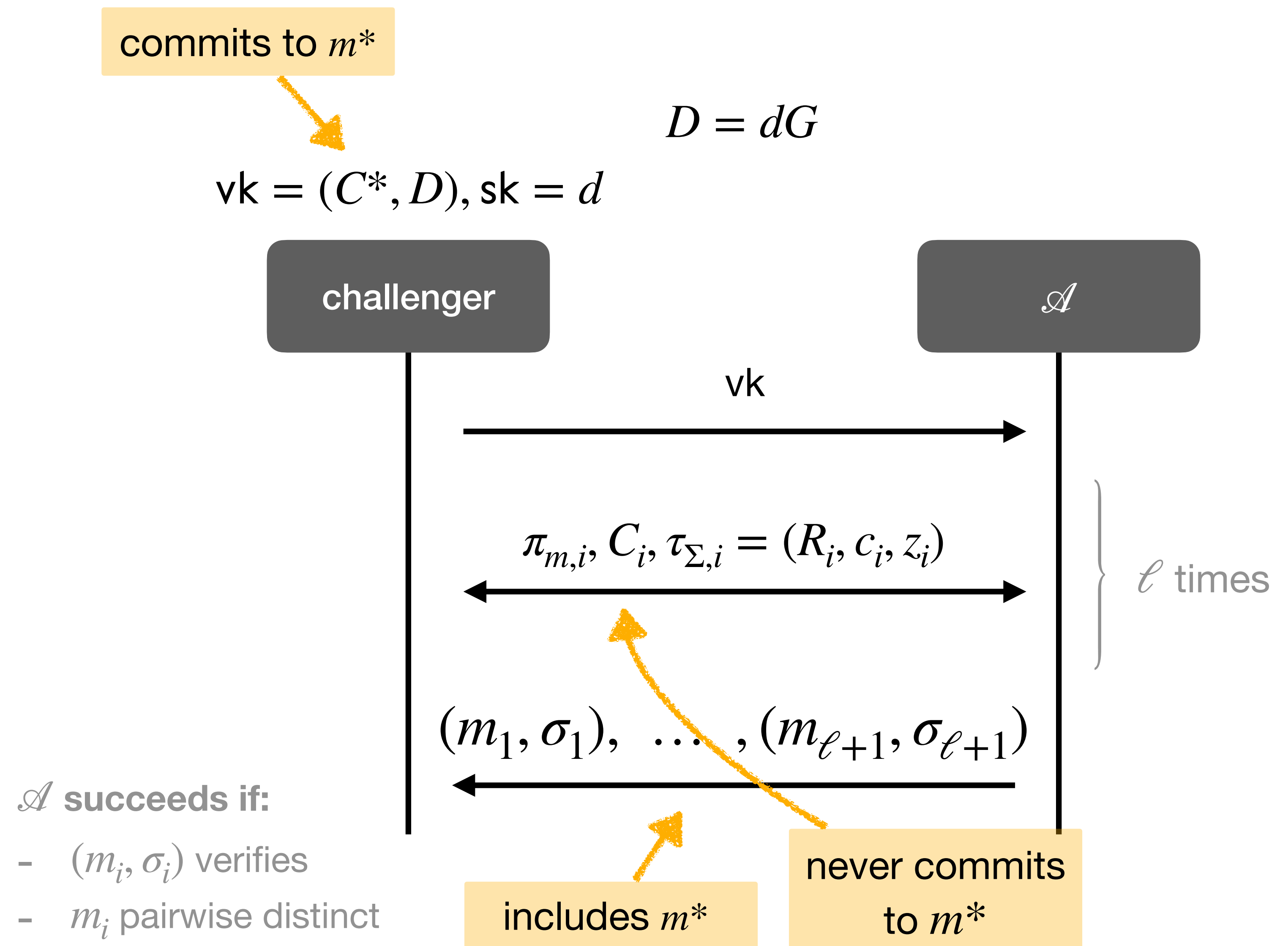
- **Step 1:** puncture vk on some message m^*
 - force adversary to provide forgery for m^*
 - never sign m^* in simulation
 - commit to m^* in C^*



Our Approach [KLR26]

Unforgeability

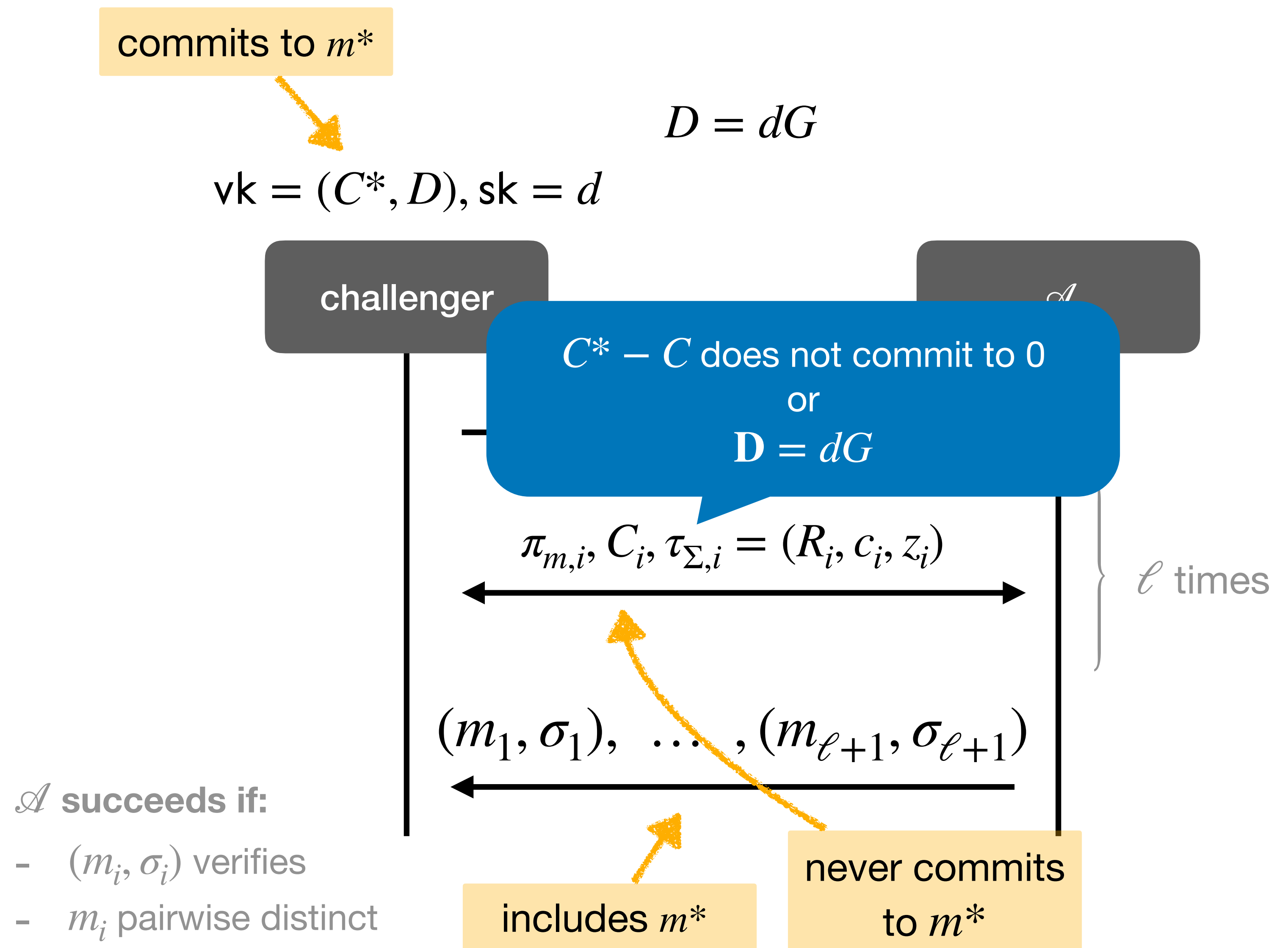
- **Step 2:** prove inequality branch honestly



Our Approach [KLR26]

Unforgeability

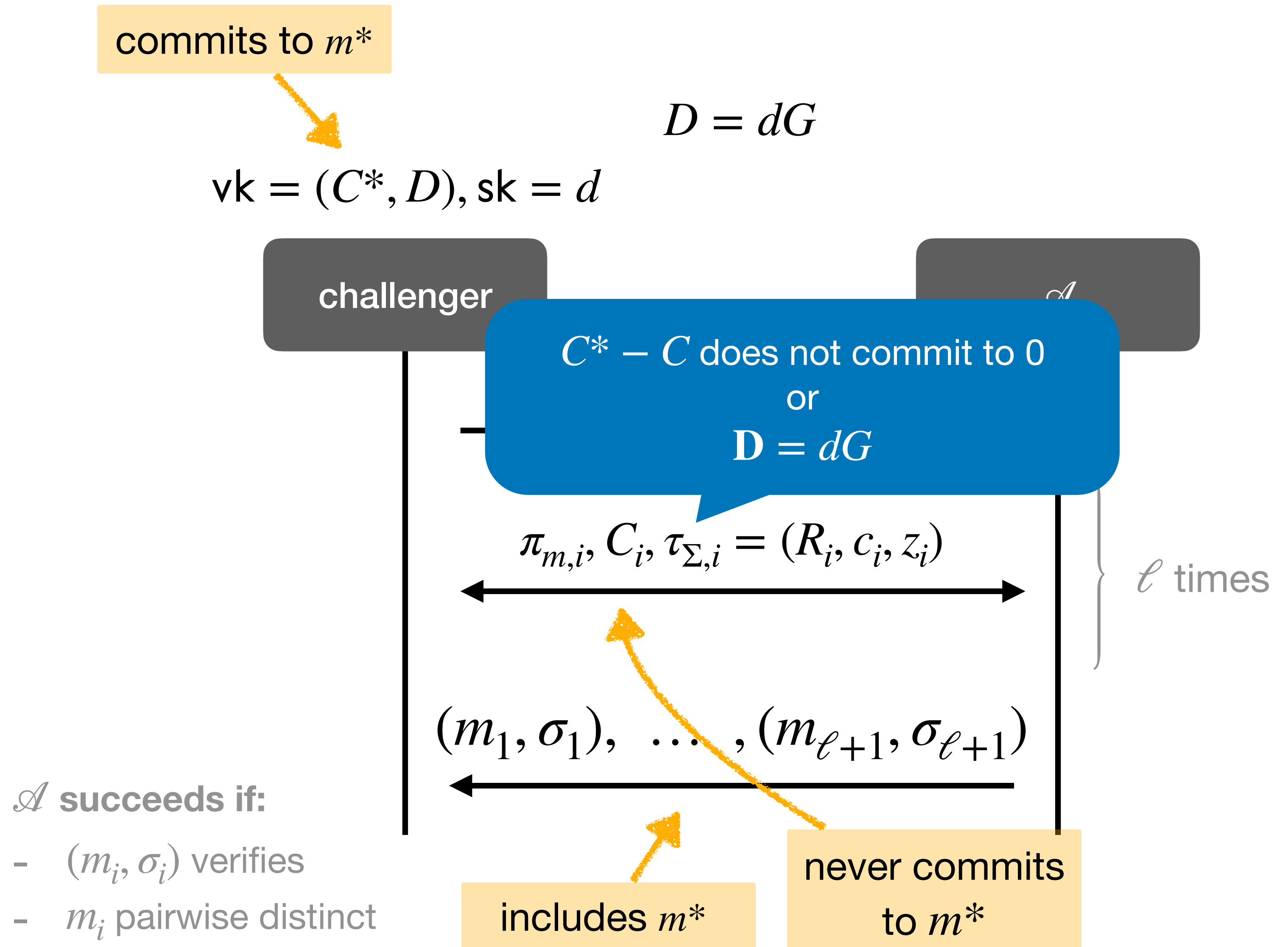
- **Step 2:** prove inequality branch honestly



Our Approach [KLR26]

Unforgeability

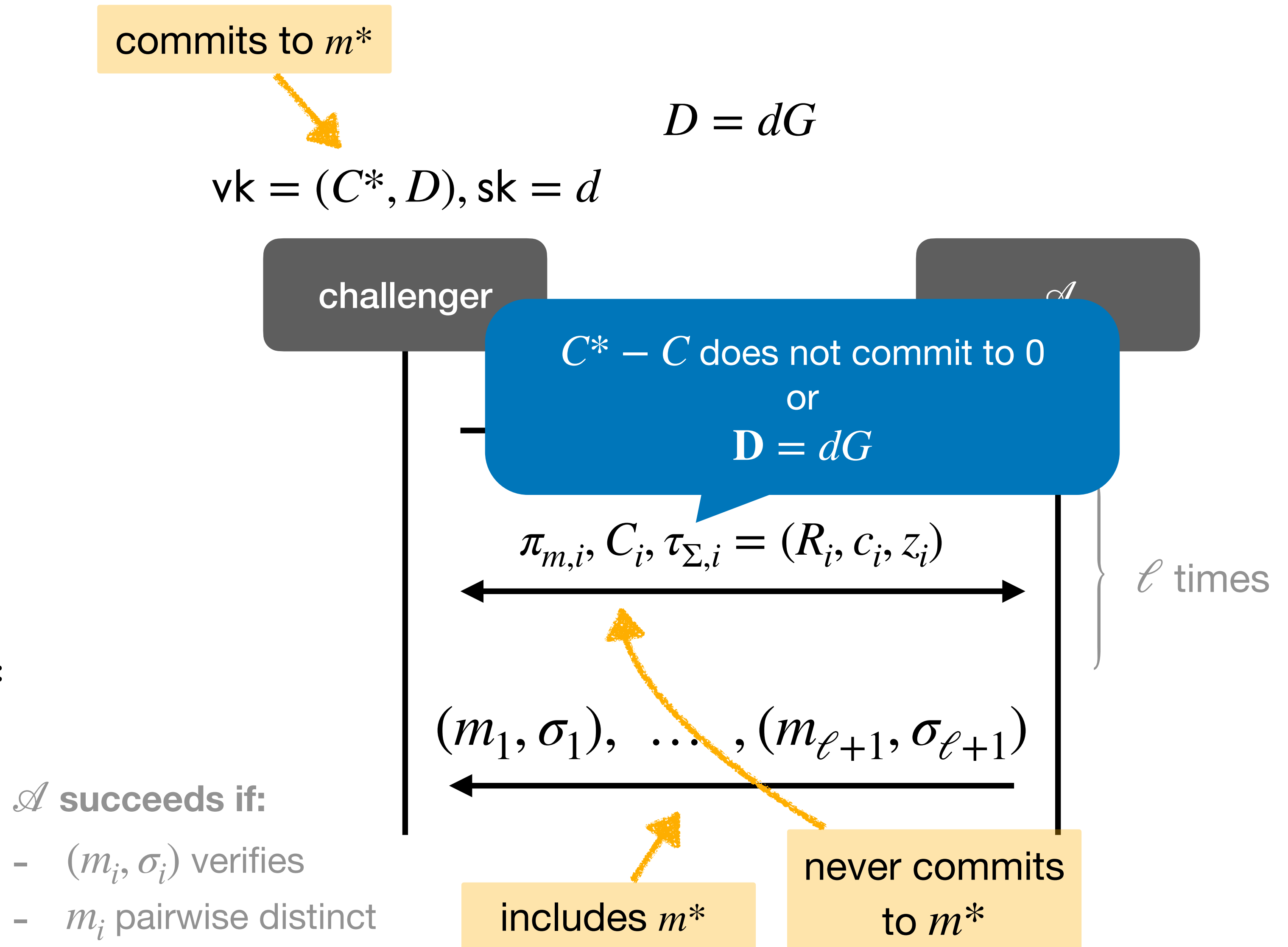
- **Step 2:** prove inequality branch honestly
 - extract to-be-signed m_i from proof $\pi_{m,i}$



Our Approach [KLR26]

Unforgeability

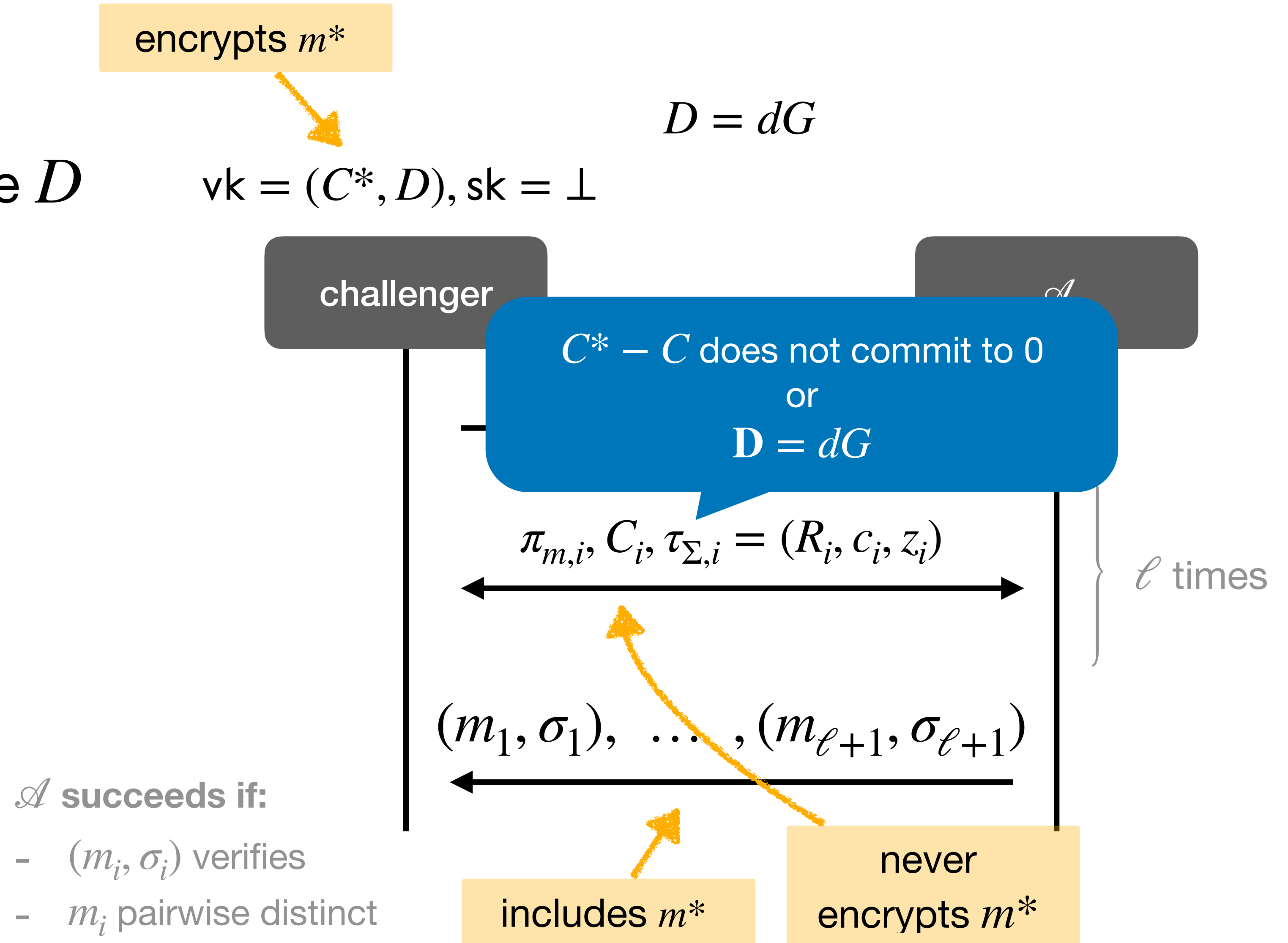
- **Step 2:** prove inequality branch honestly
 - extract to-be-signed m_i from proof $\pi_{m,i}$
 - compute witness to prove that $C^* - C$ does not **commit** to 0, i.e., $m_i \neq m^*$



Our Approach [KLR26]

Unforgeability

- **Step 3:** forget d and simulate D branch



Our Approach [KLR26]

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Com}(m^*; 0)$ commits to non-zero message
 - or $D = dG$
 - both claims are *true*

$\mathcal{A}$ succeeds if:

- (m_i, σ_i) verifies
- m_i pairwise distinct

encrypts m^*

$$D = dG$$

$$\text{vk} = (C^*, D), \text{sk} = \perp$$

challenger

$C^* - C$ does not commit to 0
or
 $D = dG$

$$\pi_{m,i}, C_i, \tau_{\Sigma,i} = (R_i, c_i, z_i)$$

ℓ times

$$(m_1, \sigma_1), \dots, (m_{\ell+1}, \sigma_{\ell+1})$$

includes m^*

never
encrypts m^*

Our Approach [KLR26]

Unforgeability

- **Conclusion:** adversary provides forgery on message m^*
 - forgery includes proof that
 - either $C^* - \text{Com}(m^*; 0)$ commits to non-zero message
 - or $D = dG$
 - both claims are *true*
- **Core Observation:** can embed DL challenge in both branches
 - in public parameters for Com and verification key D
 - simulation as before
 - rewind to extract *some* witness from the OR-proof
 - any witness breaks DL

Summary

- **New Techniques for Blind Signatures**

Summary

- **New Techniques for Blind Signatures**
 - adapt statement proven by Σ -protocol

Summary

- **New Techniques for Blind Signatures**
 - adapt statement proven by Σ -protocol
 - prove unforgeability via puncturing

Summary

- **New Techniques for Blind Signatures**
 - adapt statement proven by Σ -protocol
 - prove unforgeability via puncturing
 - rewinding-based analysis works

Summary

- **New Techniques for Blind Signatures**
 - adapt statement proven by Σ -protocol
 - prove unforgeability via puncturing
 - rewinding-based analysis works
- **Implications:**

Summary

- **New Techniques for Blind Signatures**
 - adapt statement proven by Σ -protocol
 - prove unforgeability via puncturing
 - rewinding-based analysis works
- **Implications:**
 - Blind signatures from DL in the ROM